西安電子科技大學
XIDIAN UNIVERSITY

Dimensionality Reduction and Metric Learning

Machine Learning (Bilingual)

李阳

(liy@xidian.edu.cn)

School of Artificial Intelligence

Xidian University

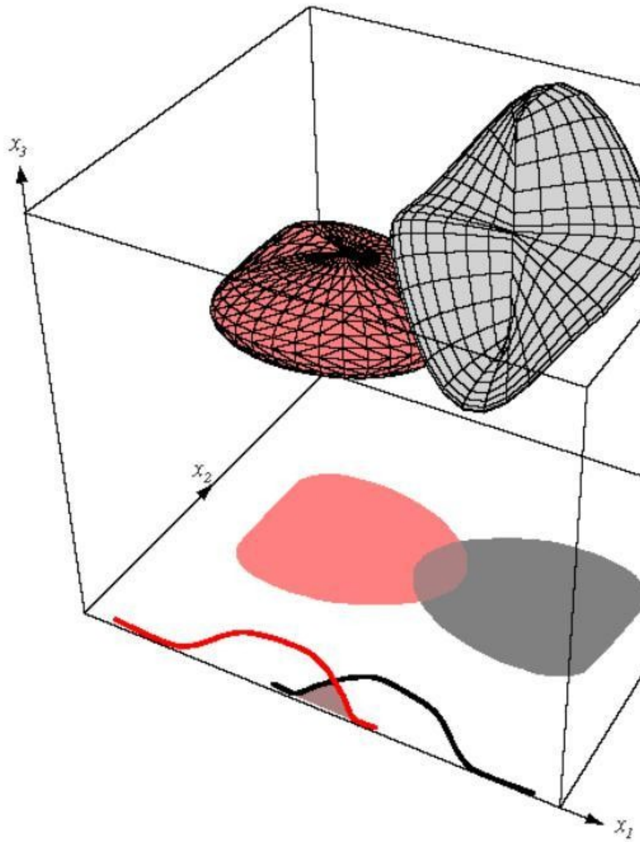
2025年10月

西安电子科技大学
XIDIAN UNIVERSITY

Contents

Introduction

- In practical multicategory applications, it is not unusual to encounter problems involving tens or hundreds of features.
- Intuitively, it may seem that each feature is useful for at least some of the discriminations.
- In general, if the performance obtained with a given set of features is inadequate, it is natural to consider adding new features.
- Even though increasing the number of features increases the complexity of the classifier, it may be acceptable for an improved performance.



There is a non-zero Bayes error in the one-dimensional x_1 space or the two-dimensional x_1, x_2 space. However, the Bayes error vanishes in the x_1, x_2, x_3 space because of non-overlapping densities.

Problems of Dimensionality

- Unfortunately, it has frequently been observed in practice that, beyond a certain point, adding new features leads to worse rather than better performance.
- This is called the **curse of dimensionality**.
- There are two issues that we must be careful about:
  - How is the classification accuracy affected by the dimensionality (relative to the amount of training data)?
  - How is the complexity of the classifier affected by the dimensionality?

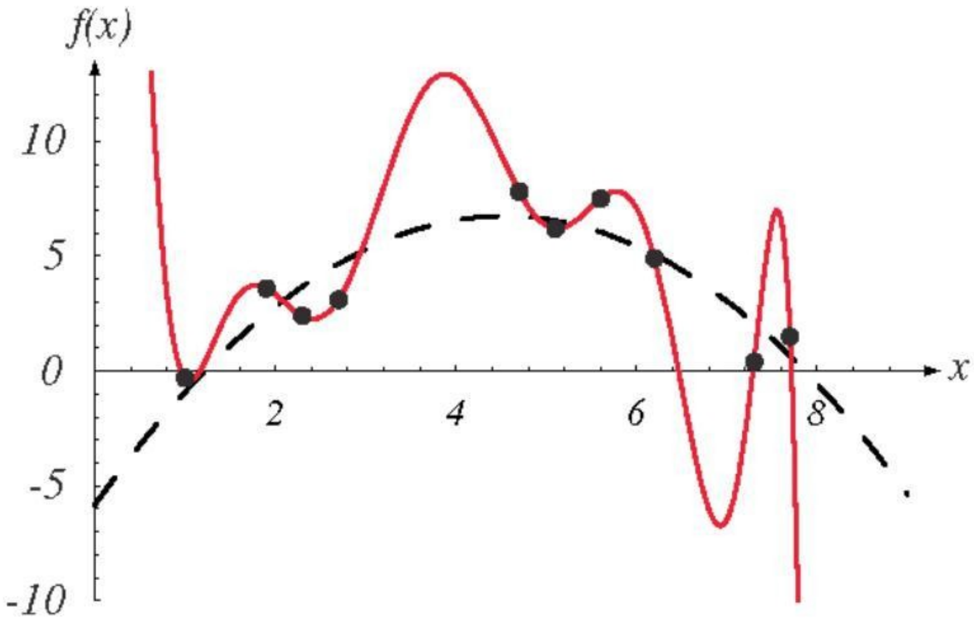
Problems of Dimensionality

- Potential reasons for increase in error include
  - wrong assumptions in model selection,
  - estimation errors due to the finite number of training samples for high-dimensional observations (overfitting).
- Potential solutions include
  - reducing the dimensionality,
  - simplifying the estimation.

Problems of Dimensionality

- Dimensionality can be reduced by
  - redesigning the features,
  - selecting an appropriate subset among the existing features,
  - combining existing features.
- Estimation errors can be simplified by
  - assuming equal covariance for all classes (for the Gaussian case),
  - using regularization,
  - using prior information and a Bayes estimate,
  - using heuristics such as conditional independence,
  - ....

Problems of Dimensionality



Problem of insufficient data is analogous to problems in curve fitting. The training data (black dots) are selected from a quadratic function plus Gaussian noise. A tenth-degree polynomial fits the data perfectly but we prefer a second-order polynomial for better generalization.

Problems of Dimensionality

- All of the commonly used classifiers can suffer from the curse of dimensionality.
- While an exact relationship between the probability of error, the number of training samples, the number of features, and the number of parameters is very difficult to establish, some guidelines have been suggested.
- It is generally accepted that using at least ten times as many training samples per class as the number of features $(n/d > 10)$ is a good practice.
- The more complex the classifier, the larger should the ratio of sample size to dimensionality be.

Feature Reduction

- One way of coping with the problem of high dimensionality is to reduce the dimensionality by combining features.
- Issues in feature reduction:
  - Linear vs. non-linear transformations.
  - Use of class labels or not (depends on the availability of training data).
  - Training objective:
    - minimizing classification error (discriminative training),
    - minimizing reconstruction error (PCA),
    - maximizing class separability (LDA),
    - retaining interesting directions (projection pursuit),
    - making features as independent as possible (ICA),
    - embedding to lower dimensional manifolds (Isomap, LLE).

- Linear combinations are particularly attractive because they are simple to compute and are analytically tractable.
- Linear methods project the high-dimensional data onto a lower dimensional space.
- Advantages of these projections include
  - reduced complexity in estimation and classification,
  - ability to visually examine the multivariate data in two or three dimensions.

Feature Reduction

- Given $\mathbf{x} \in \mathbb{R}^d$, the goal is to find a linear transformation $\mathbf{A}$ that gives $\mathbf{y} = \mathbf{A}^T\mathbf{x} \in \mathbb{R}^{d'}$ where $d' < d$.
- Two classical approaches for finding optimal linear transformations are:
  - **Principal Components Analysis (PCA)**: Seeks a projection that best represents the data in a least-squares sense.
  - **Linear Discriminant Analysis (LDA)**: Seeks a projection that best separates the data in a least-squares sense.

Principal Component Analysis

Principal Component Analysis (PCA)

Principal Components Analysis

- Given $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, the goal is to find a d'-dimensional subspace where the reconstruction error of $\mathbf{x}_i$ in this subspace is minimized.
- The criterion function for the reconstruction error can be defined in the least-squares sense as

$$J_{d'} = \sum_{i=1}^{n} \left\| \sum_{k=1}^{d'} \mathbf{y}_{ik}\mathbf{e}_k - \mathbf{x}_i \right\|^2$$

where $\mathbf{e}_1, \dots, \mathbf{e}_{d'}$ are the bases for the subspace (stored as the columns of $\mathbf{A}$) and $\mathbf{y}_i$ is the projection of $\mathbf{x}_i$ onto that subspace.

[1]

[1] C.M. Bishop, Ch. 12, *Continuous Latent Variables*, **Pattern Recognition and Machine Learning**. Singapore: Springer, 2006.

主成分分析

- 最小化重建误差

$$J = \frac{1}{N}\sum_{n=1}^{N}\|x_n - \tilde{x}_n\|^2$$

其中 $\{u_i\}, i=1,\ldots,D$ 表示正交基向量，且 $u_i^T u_j = \delta_{ij}$。

则 $x_n = \sum_{i=1}^{D}(x_n^T u_i)u_i$ 可近似表示为

$$\tilde{x}_n = \sum_{i=1}^{M} z_{ni}u_i + \sum_{i=M+1}^{D} b_i u_i$$

其中 z_{ni} 取决于数据，且 b_i 为常数项。

主成分分析

- 计算主成分分析的偏导数 zni 且 b_i：

$$\tilde{x}_n = \sum_{i=1}^{M} z_{ni} u_i + \sum_{i=M+1}^{D} b_i u_i \rightarrow J = \frac{1}{N}\sum_{n=1}^{N} ||x_n - \tilde{x}_n||^2$$

偏导数

导数

$$\begin{cases} z_{ni} = x_n^T u_i,\ i = 1, \dots, M \\ b_i = \bar{x}^T u_i,\ i = M+1, \dots, D \end{cases}$$

$$x_n - \tilde{x}_n = \sum_{i=M+1}^{D} \{(x_n - \bar{x})^T u_i\} u_i$$

$$\Rightarrow J = \frac{1}{N}\sum_{n=1}^{N}\sum_{i=M+1}^{D} (x_n^T u_i - \bar{x}^T u_i)^2 = \sum_{i=M+1}^{D} u_i^T S u_i$$

其中 $$S = \frac{1}{N}\sum_{n=1}^{N}(x_n - \bar{x})(x_n - \bar{x})^T$$

Intuitions

- Consider the case of a 2D data space $d = 2$ and a 1D principal subspace $d' = 1$.
- We need to choos a direction $\mathbf{u}_2$ so as to minimize $J = \mathbf{u}_2^T \mathbf{S} \mathbf{u}_2$, subject to the constraint $\mathbf{u}_2^T \mathbf{u}_2 = 1$.
- Using a Lagrange multiplier λ_2, we consider

$$\tilde{J} = \mathbf{u}_2^T \mathbf{S} \mathbf{u}_2 + \lambda_2 (1 - \mathbf{u}_2^T \mathbf{u}_2).$$

- Setting the derivative w.r.t. $\mathbf{u}_2$ to zero, we have

$$\mathbf{S} \mathbf{u}_2 = \lambda_2 \mathbf{u}_2.$$

$$\Rightarrow \quad J = \sum_{i=M+1}^{D} \lambda_i$$

主成分分析

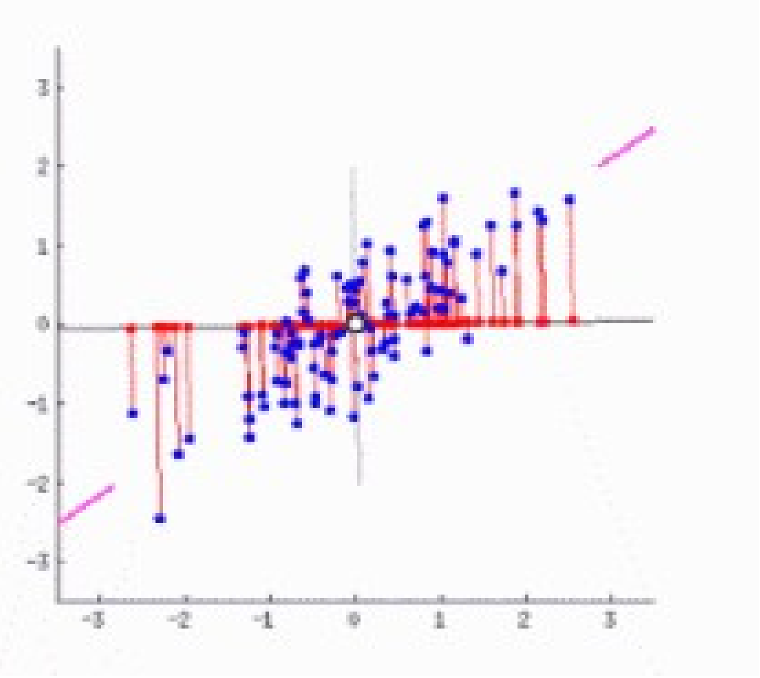
最大方差公式

- 投影后的方差：

$$\frac{1}{N}\sum_{n=1}^{N}\{u_1^T x_n - u_1^T \bar{x}\}^2 = u_1^T S u_1$$

其中

$$S = \frac{1}{N}\sum_{n=1}^{N}(x_n - \bar{x})(x_n - \bar{x})^T$$



Maximum variance formulation

- We now maximize the projected variance $\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$ w.r.t. $\mathbf{u}_1$.
- To prevent $\|\mathbf{u}_1\| \to \infty$, we add a normalization constraint $\mathbf{u}_1^T \mathbf{u}_1 = 1$.
- Introducing Lagrange multiplier λ_1, we need to maximize

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 + \lambda_1 (1 - \mathbf{u}_1^T \mathbf{u}_1)..$$

- Let the derivative w.r.t. $\mathbf{u}_1$ equal to zero, we have

$$\mathbf{S}\mathbf{u}_1 = \lambda_1 \mathbf{u}_1.$$

- 方差: $$\frac{1}{N}\sum_{n=1}^{N}\{u_1^T x_n - u_1^T \bar{x}\}^2 = u_1^T S u_1$$ $(=)\lambda_1$

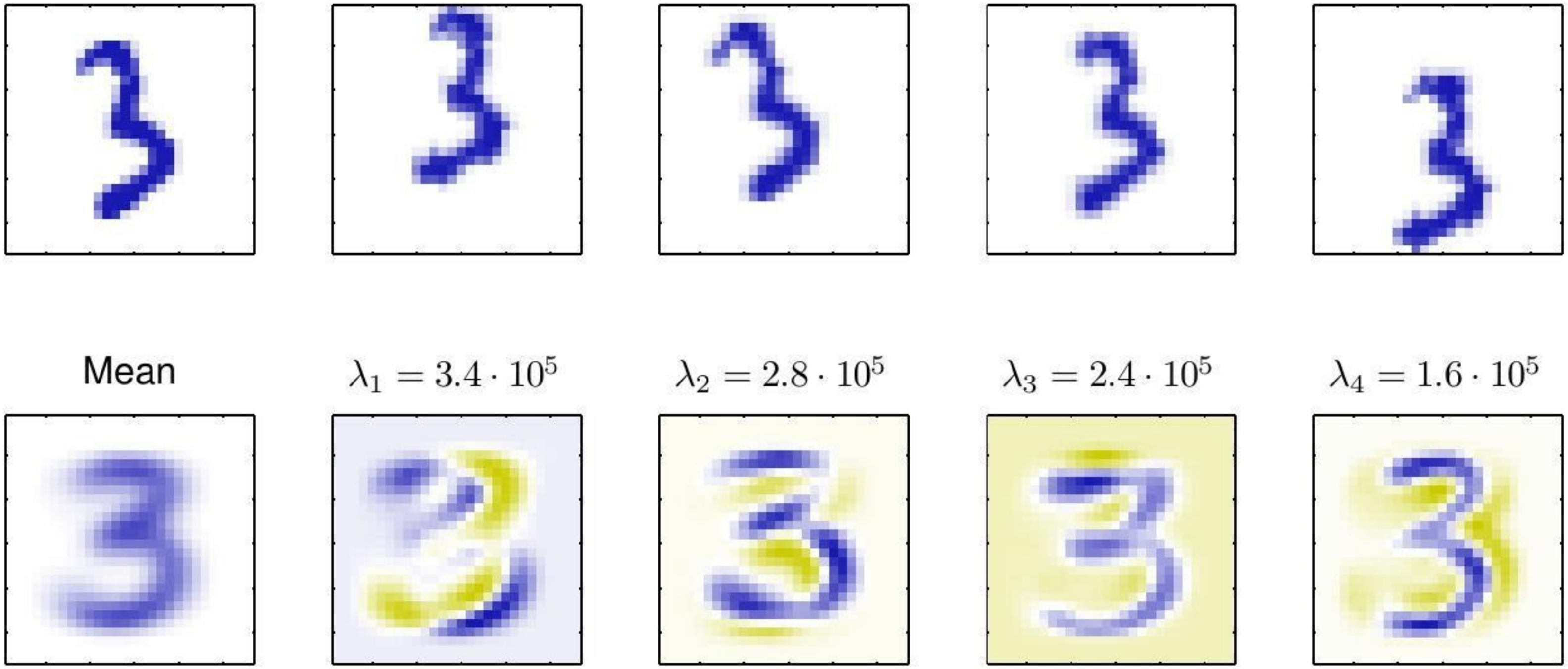
λ值越大，方差越大。

Signal Processing Perspective

- Often there will be just a few large eigenvalues, and this implies that the d'-dimensional subspace contains the signal and the remaining $d - d'$ dimensions generally contain noise.

- The actual subspace where the data may lie is related to the **intrinsic dimensionality** that determines whether the given d-dimensional patterns can be described adequately in a subspace of dimensionality less than d.
- The geometric interpretation of intrinsic dimensionality is that the entire data set lies on a topological d'-dimensional hypersurface.
- Note that the intrinsic dimensionality is not the same as the linear dimensionality which is related to the number of significant eigenvalues of the scatter matrix of the data.

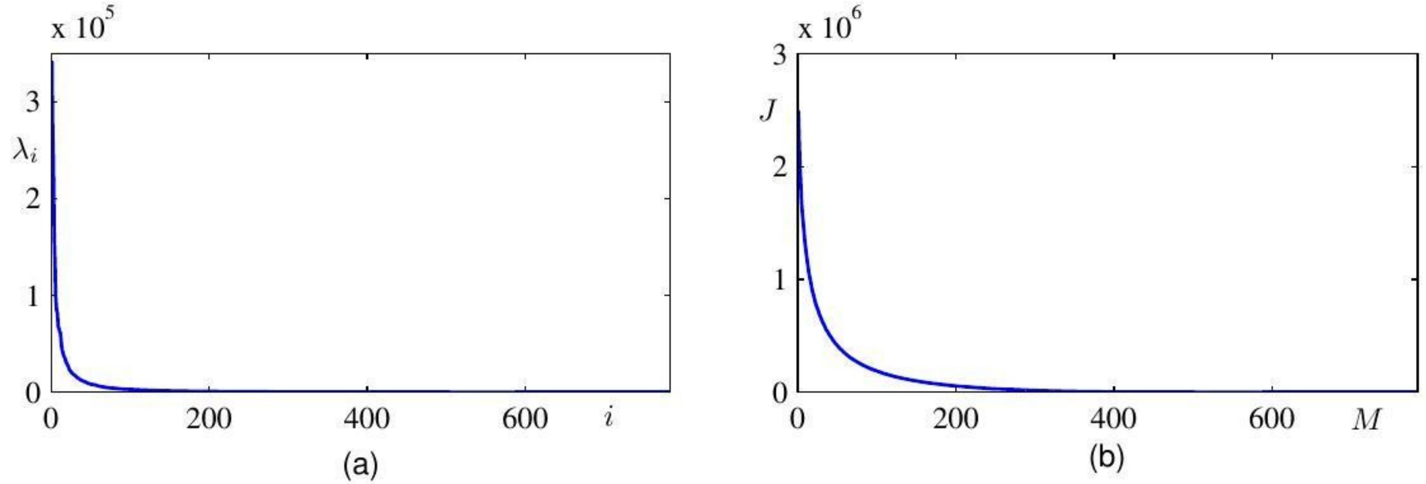
PCA Examples



PCA for data compression

The mean vector $\bar{\mathbf{x}}$ along with the first four PCA eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_4$ for the off-line digits data set, together with the corresponding eigenvalues.

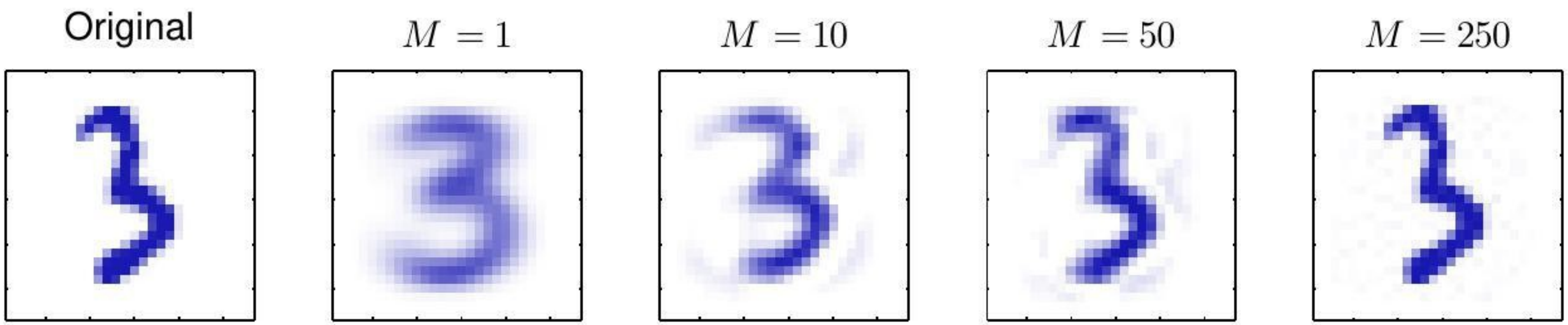
PCA Examples



(a) Plot of the eigenvalue spectrum for the off-line digits data set.

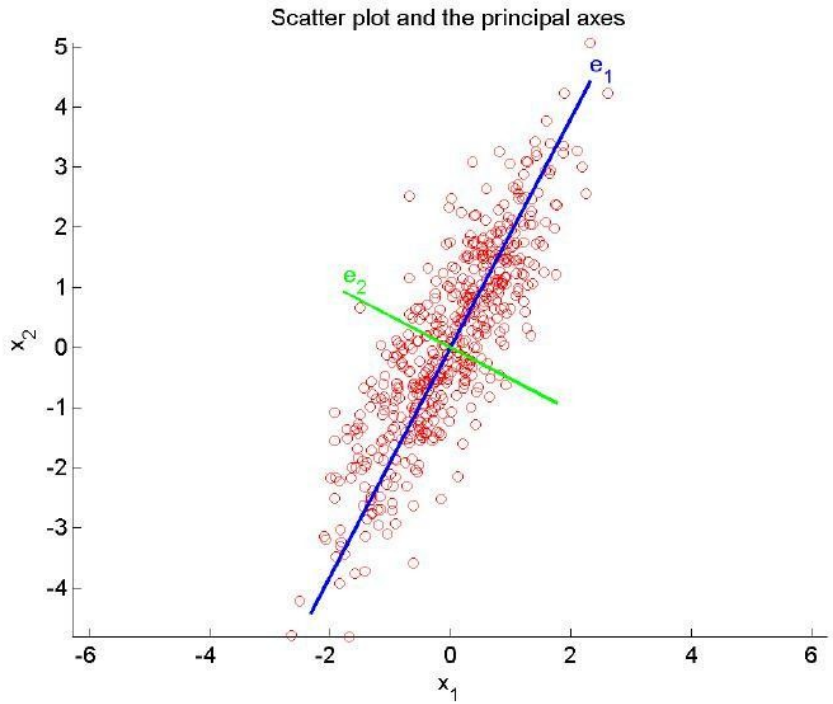
(b) Plot of the sum of the discarded eigenvalues, which represents the sum-of-squares distortion J introduced by projecting the data onto a principal component subspace of dimensionality M.

PCA Examples

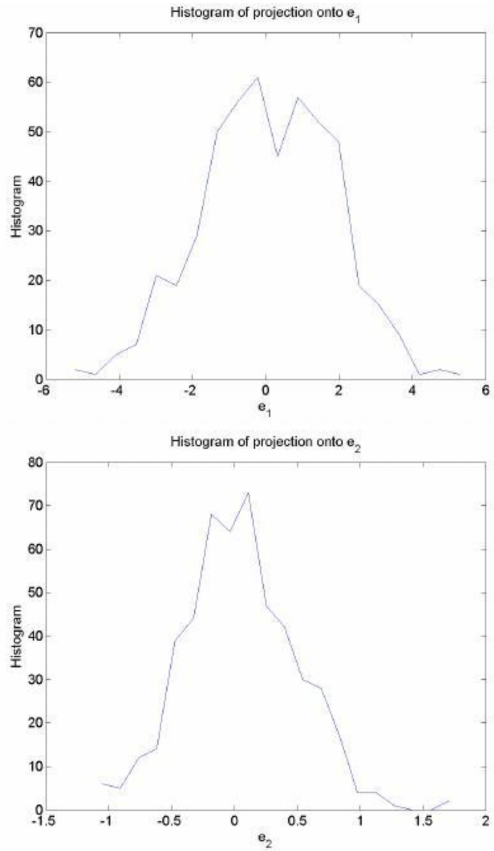


An original example from the off-line digits data set together with its PCA reconstructions obtained by retaining M principal components for various values of M. As M increases the reconstruction becomes more accurate and would become perfect when $M = d = 28 \times 28 = 784$.

PCA Examples

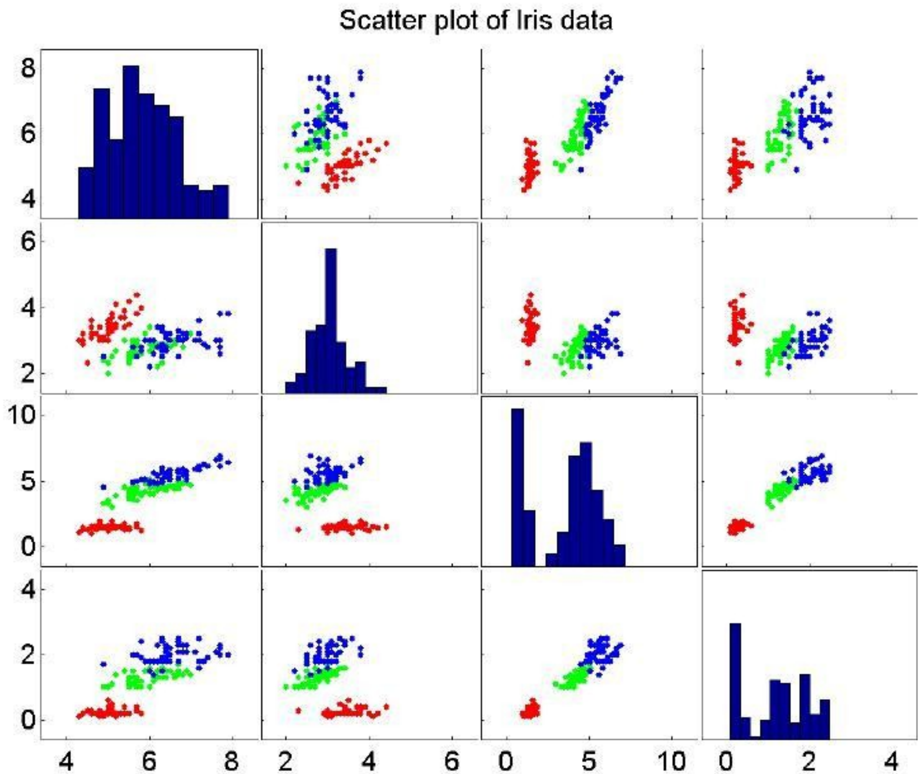


(a) Scatter plot. Scatter plot (red dots) and the principal axes for a bivariate sample. The blue line shows the axis e_1 with the greatest variance and the green line shows the axis e_2 with the smallest variance. Features are now uncorrelated.



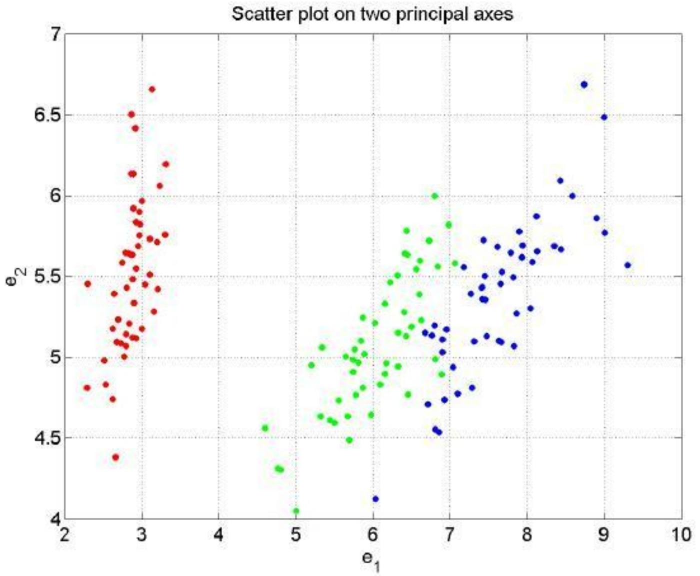
(b) Projection onto e_1.
(c) Projection onto e_2.

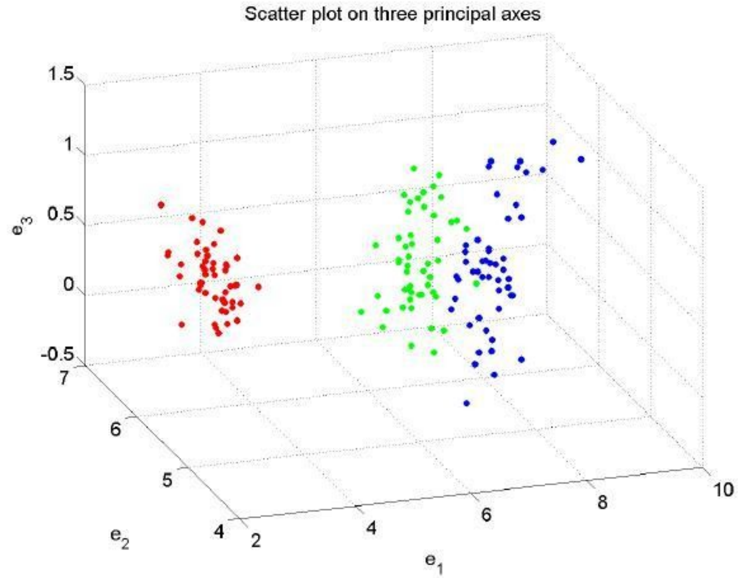
PCA Examples



Scatter plot of the iris data. Diagonal cells show the histogram for each feature. Other cells show scatters of pairs of features x_1, x_2, x_3, x_4 in top-down and left-right order. Red, green and blue points represent samples for the setosa, versicolor and virginica classes, respectively.

PCA Examples





Scatter plot of the projection of the iris data onto the first two and the first three principal axes. Red, green and blue points represent samples for the setosa, versicolor and virginica classes, respectively.

Probabilistic PCA

Probabilistic PCA

Probabilistic PCA represents a constrained form of the Gaussian distribution in which the number of free parameters can be restricted while still allowing the model to capture the dominant correlations in a data set.

- Probabilistic PCA can be used to model class-conditional densities and hence be applied to classification problems.
- The probabilistic PCA model can be run generatively to provide samples from the distribution.
- Probabilistic PCA is a simple example of the **linear-Gaussian** framework, in which all of the marginal and conditional distributions are Gaussian.

Probabilistic PCA

- Let us first introducing an **explicit latent variable** $\mathbf{z}$ corresponding to the principal-component subspace.
- Next we define a Gaussian prior distribution $p(\mathbf{z})$ over the latent variable,

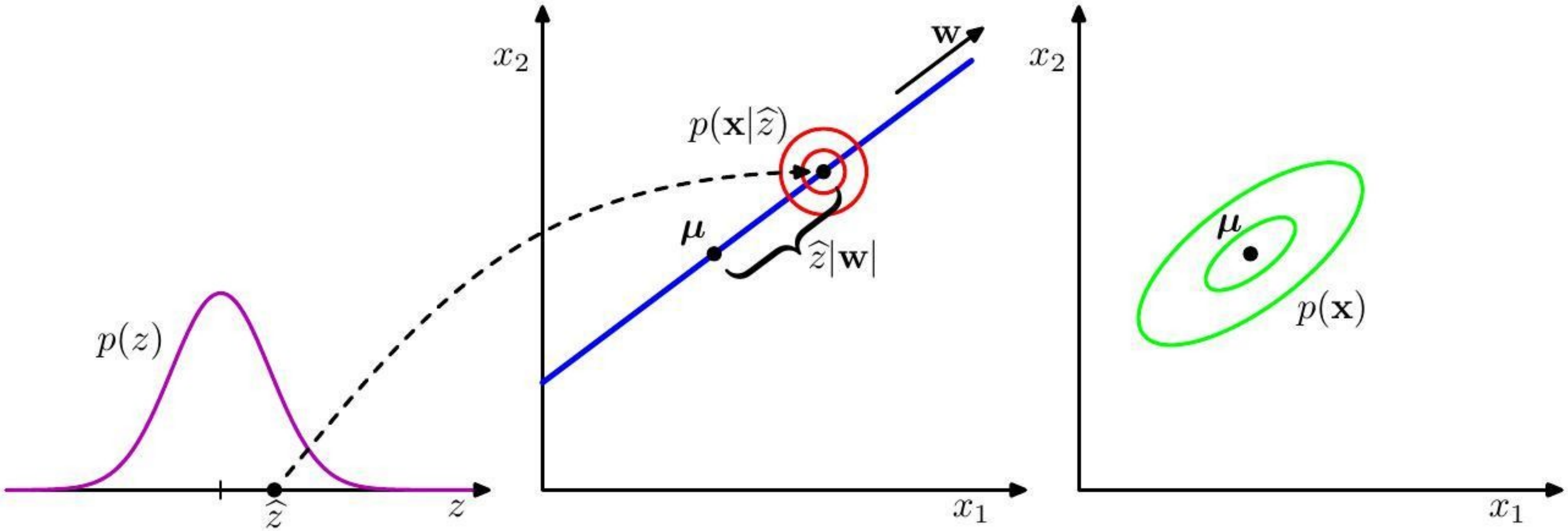
$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I}),$$

- together with a Gaussian conditional distribution $p(\mathbf{x}|\mathbf{z})$ for the observed variable x conditioned on the value of the latent variable.

$$p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- We can view the probabilistic PCA model from a generative viewpoint, so that

$$\mathbf{x} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}.$$



An illustration of the generative view of the probabilistic PCA model for a 2D data space and a 1D latent space. An observed data point $\mathbf{x}$ is generated by first drawing a value $\widehat{z}$ for the latent variable from its prior distribution $p(\widehat{z})$ and then drawing a value for $\mathbf{x}$ from an isotropic Gaussian distribution (illustrated by the red circles) having mean $\mathbf{w}\widehat{z} + \boldsymbol{\mu}$ and covariance $\sigma^2\mathbf{I}$. The green ellipses show the density contours for the marginal distribution $p(\mathbf{x})$.

- Suppose we wish to determine the values of the parameters $\mathbf{W}$, $\boldsymbol{\mu}$ and σ^2 using maximum likelihood.
- We need an expression for the marginal distribution $p(\mathbf{x})$,

$$p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z}) \, \mathrm{d}\mathbf{z} = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}, \mathbf{C}),$$

where the $d \times d$ covariance matrix $\mathbf{C}$ is defined by

$$\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2 \mathbf{I}.$$

- When we evaluate the predictive distribution, we require $\mathbf{C}^{-1}$, which involves the inversion of a $d \times d$ matrix.
- Using the matrix inversion identity, we have

$$\mathbf{C}^{-1} = \sigma^{-2}\mathbf{I} - \sigma^{-2}\mathbf{W}\mathbf{M}^{-1}\mathbf{W}^T,$$

where the $d' \times d'$ matrix $\mathbf{M}$ is defined by

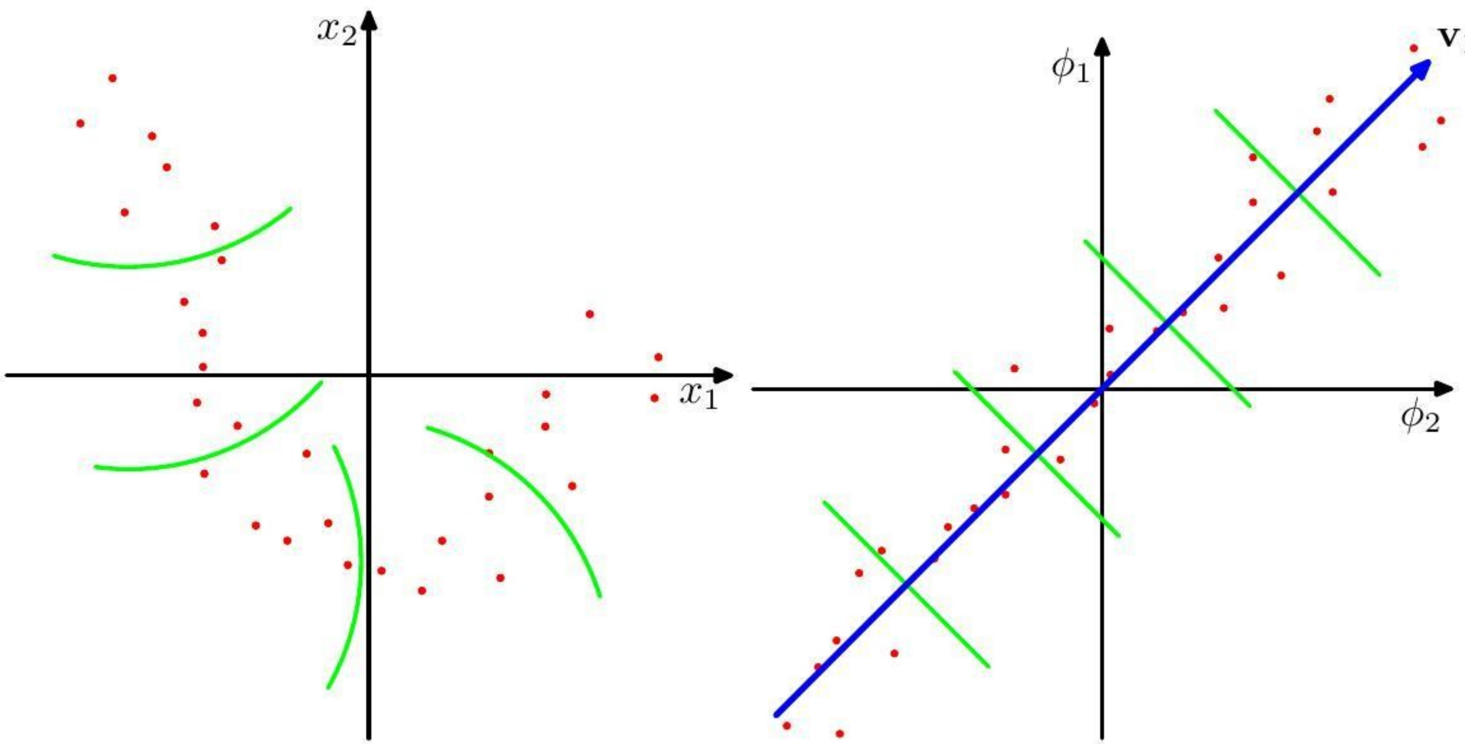
$$\mathbf{M} = \mathbf{W}^T\mathbf{W} + \sigma^2\mathbf{I}.$$

- The posterior distribution $p(\mathbf{z}|\mathbf{x})$ is given by

$$p(\mathbf{z}|\mathbf{x}) = \mathcal{N}\big(\mathbf{z}|\mathbf{M}^{-1}\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu}), \sigma^{-2}\mathbf{M}\big).$$

Kernel PCA

- Now consider a nonlinear transformation $\phi(\mathbf{x})$, and we can perform standard PCA in the feature space.



Kernel PCA

- Recall that PCA are defined by the eigenvectors $\mathbf{u}_i$ of the covariance matrix

$$\mathbf{S}\mathbf{u}_i = \lambda_i \mathbf{u}_i \qquad \mathbf{S} = \frac{1}{N}\sum_{n=1}^{N} \mathbf{x}_n \mathbf{x}_n^T.$$

- The sample covariance matrix in feature space is given by

$$\mathbf{C} = \frac{1}{N}\sum_{n=1}^{N} \phi(\mathbf{x}_n)\phi(\mathbf{x}_n)^T,$$

and its eigenvectors expansion is defined by

$$\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i.$$

Kernel PCA

- Our goal is to solve this eigenvalue problem without having to work explicitly in the feature space.
- By expressing vector $\mathbf{v}_i$ by a linear combination of the $\phi(\mathbf{x}_n)$,

$$\mathbf{v}_i = \sum_{n=1}^{N} a_{in} \phi(\mathbf{x}_n),$$

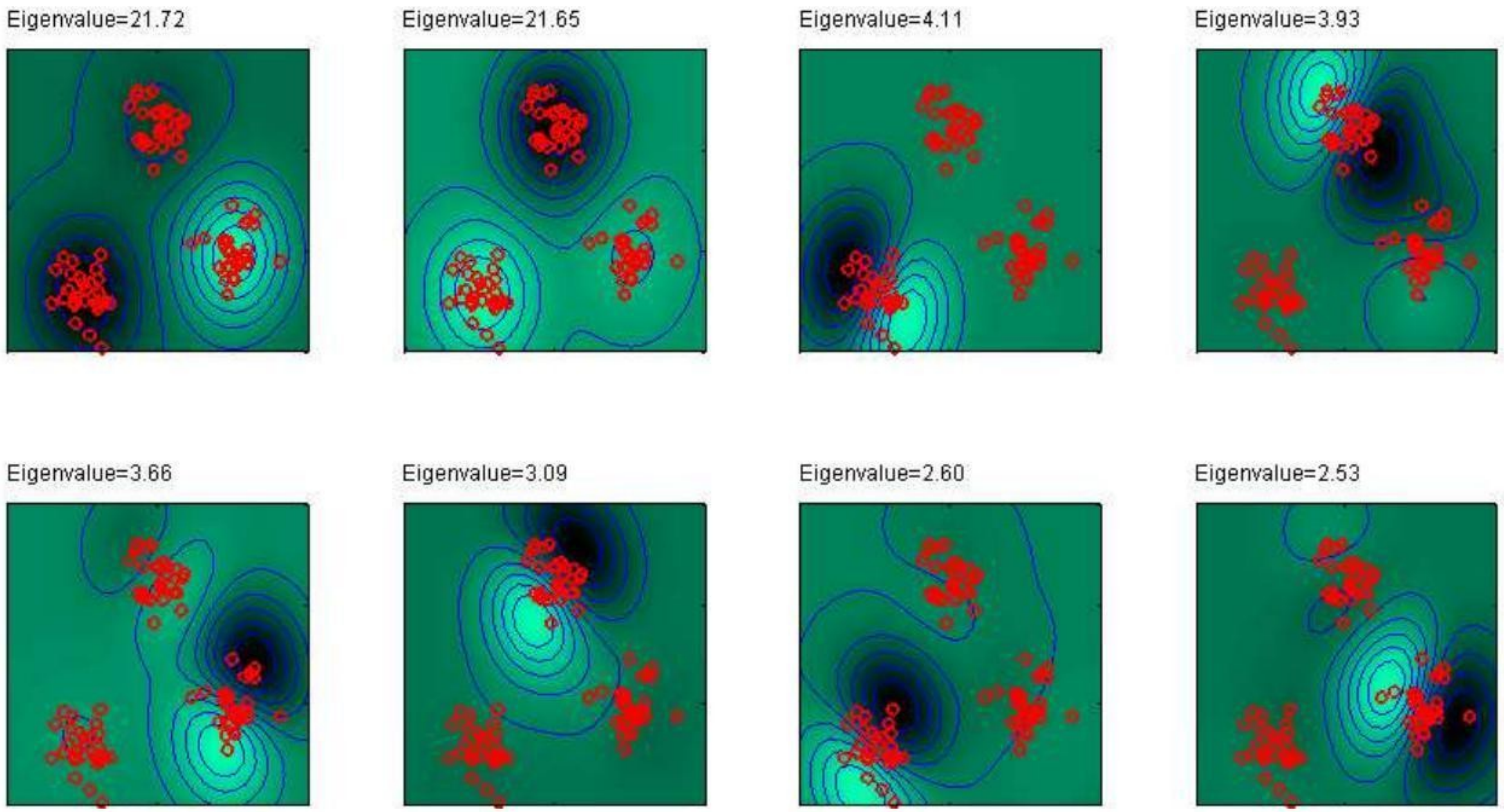
- we can find solutions for $\mathbf{a}_i$ by solving the following eigenvalue problem

$$\mathbf{K}\mathbf{a}_i = \lambda_i N \mathbf{a}_i.$$

- The projection of a point $\mathbf{x}$ onto eigenvector $\mathbf{v}_i$ is given by

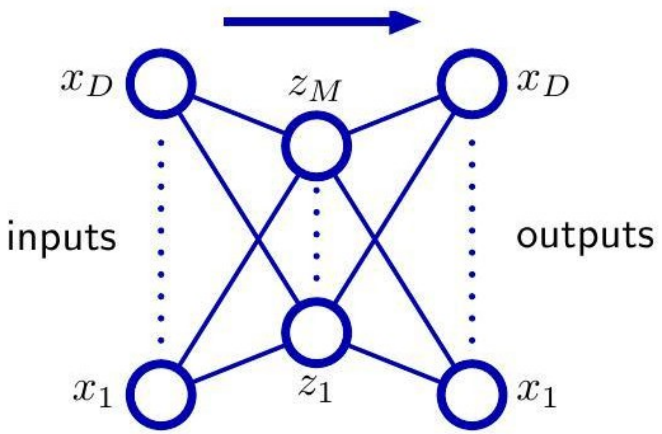
$$y_i(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{v}_i = \sum_{n=1}^{N} a_{in} k(\mathbf{x}, \mathbf{x}_n).$$

Kernel PCA Example with a Gaussian kernel



Autoencoder

自编码器 / 自关联神经网络



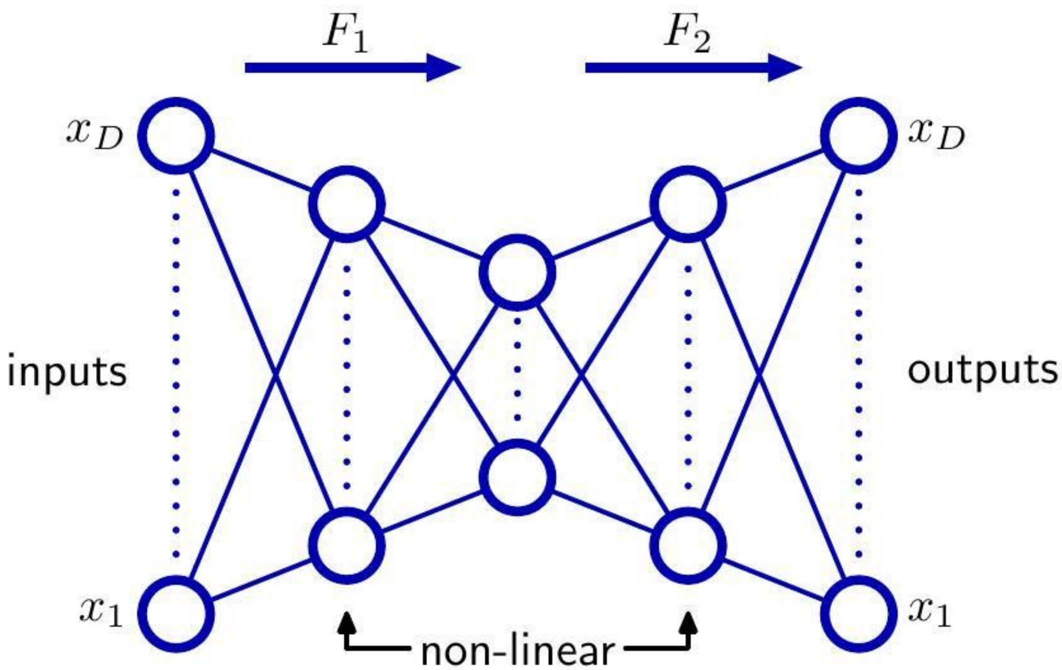
Links representing bias parameters have been omitted for clarity.

An autoassociative multilayer perceptron having two layers of weights. Such a network is trained to map input vectors onto themselves by minimization of a sum-of-squares error

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N} \|y(\mathbf{x}_n, \mathbf{w}) - \mathbf{x}_n\|^2.$$

Even with non-linear units in the hidden layer, such a network is equivalent to linear principal component analysis.

自编码器/自关联神经网络



Addition of extra hidden layers of nonlinear units gives an autoassociative network which can perform a nonlinear dimensionality reduction.

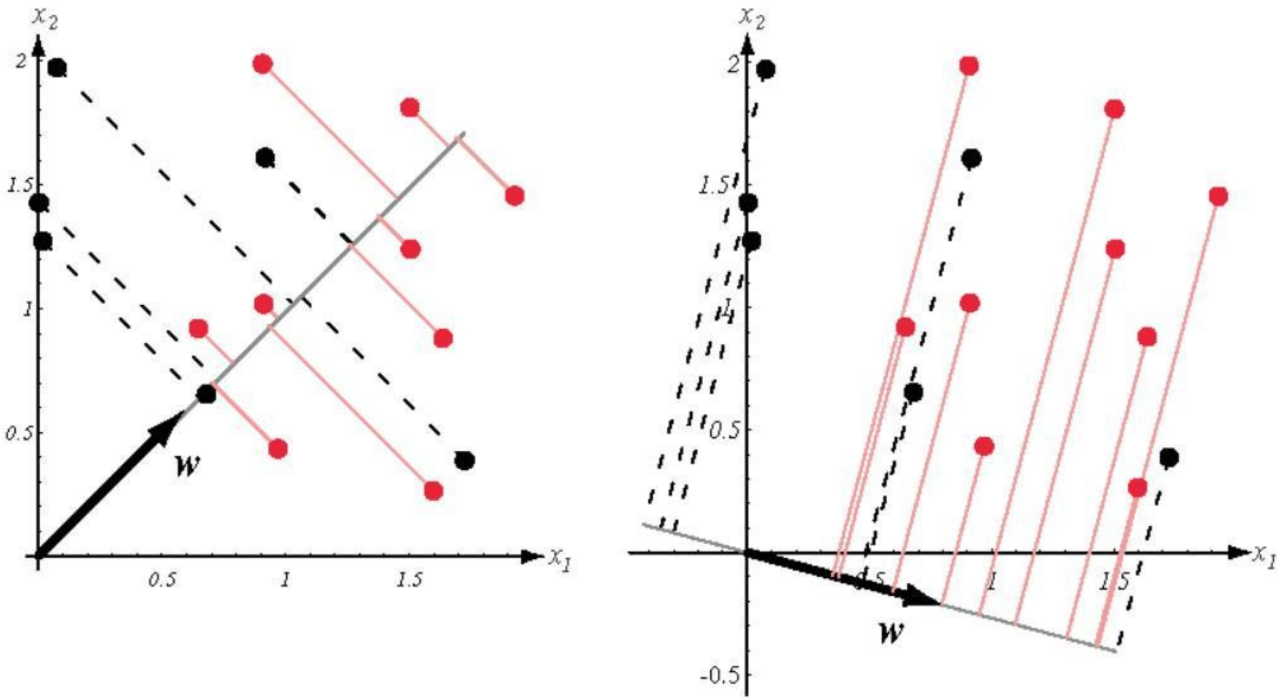
Linear Discriminant Analysis

Linear Discriminant Analysis

- Whereas PCA seeks directions that are efficient for representation, discriminant analysis seeks directions that are efficient for discrimination.
- Given $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^d$ divided into two subsets $\mathcal{D}_1$ and $\mathcal{D}_2$ corresponding to the classes $\mathcal{C}_1$ and $\mathcal{C}_2$, respectively, the goal is to find a projection onto a line defined as

$$\mathbf{y} = \mathbf{w}^T \mathbf{x}$$

where the points corresponding to $\mathcal{D}_1$ and $\mathcal{D}_2$ are well separated.



Projection of the same set of samples onto two different lines in the directions marked as w. The figure on the right shows greater separation between the red and black projected points.

Projection maximizes class separation

- Consider a two-class problem in which there are N_1 points of class $\mathcal{C}_1$ and N_2 points of class $\mathcal{C}_2$, so that the mean vectors of the two classes are given by

$$\mathbf{m}_1 = \frac{1}{N_1}\sum_{n\in\mathcal{C}_1}\mathbf{x}_n, \qquad \mathbf{m}_2 = \frac{1}{N_2}\sum_{n\in\mathcal{C}_2}\mathbf{x}_n.$$

- The simplest measure of the separation of the classes, when projected onto $\mathbf{w}$, is the separation of the projected class means.
- This suggests that we might choose $\mathbf{w}$ so as to maximize

$$m_2 - m_1 = \mathbf{w}^{\mathrm{T}}(\mathbf{m}_2 - \mathbf{m}_1)$$

where

$$m_k = \mathbf{w}^{\mathrm{T}}\mathbf{m}_k$$

is the mean of the projected data from class $\mathcal{C}_k$.

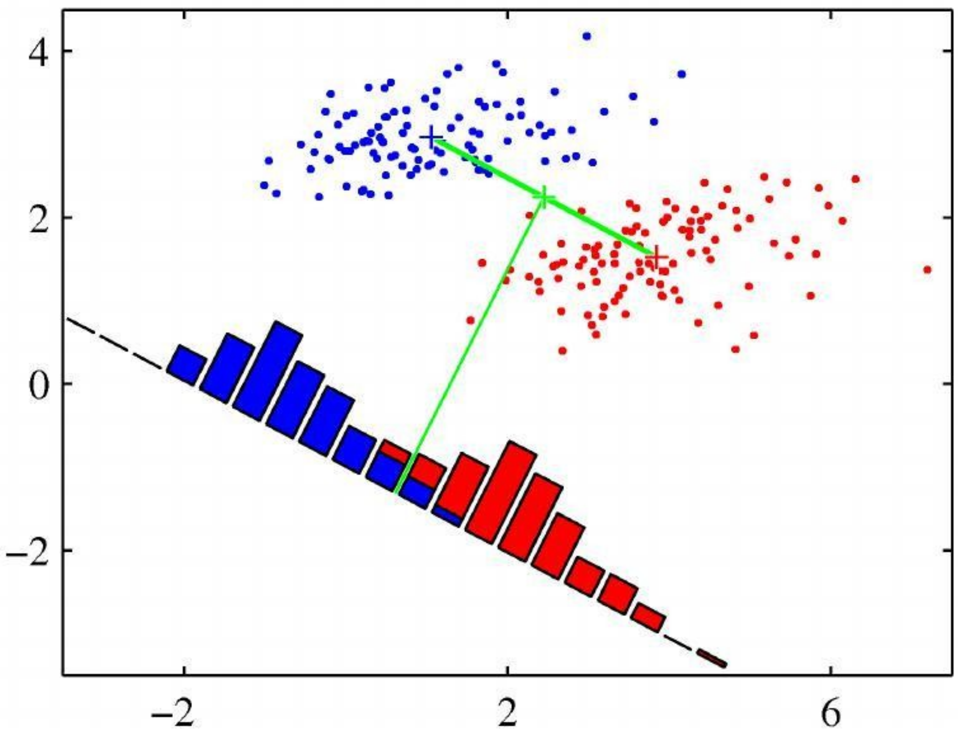
Projection maximizes class separation

- However, this expression can be made arbitrarily large simply by increasing the magnitude of $\mathbf{w}$.
- To solve this problem, we could constrain $\mathbf{w}$ to have unit length, so that $\sum_i w_i^2 = 1$.
- Using a Lagrange multiplier to perform the constrained maximization, we then find that

$$\mathbf{w} \propto (\mathbf{m}_2 - \mathbf{m}_1).$$

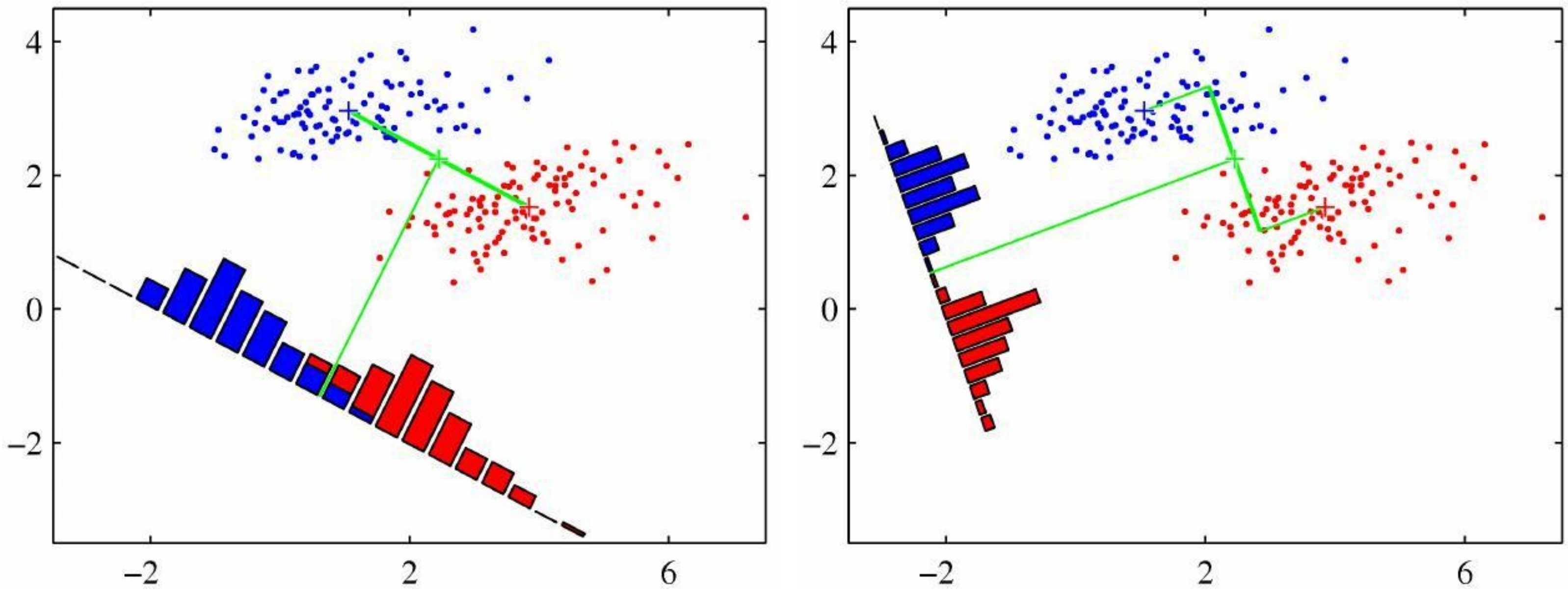
A difficult case

- A difficulty arises from the **strongly nondiagonal covariances** of the class distributions.



This shows two classes that are well separated in the original two-dimensional space (x_1, x_2) but that have considerable overlap when projected onto the line joining their means.

Fisher's linear discriminant illustrated



Finsher's linear discriminant

- The idea proposed by Fisher is to maximize a function that will **give a large separation between the projected class means** while also **giving a small variance within each class**, thereby minimizing the class overlap.
- With $y_n = \mathbf{w}^{\mathrm{T}}\mathbf{x}_n$, the **within-class variance** of the transformed data from class $\mathcal{C}_k$ is therefore given by

$$s_k^2 = \sum_{n \in \mathcal{C}_k} (y_n - m_k)^2.$$

- We can define the **total within-class varaince** for the whole data set to be simply $s_1^2 + s_2^2$.
- The **Fisher criterion** is defined to be the ratio of the **between-class variance** to the **within-class variance** and is given by

$$J(\mathbf{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2}.$$

Goal function of the projection

- We can make the dependence on $\mathbf{w}$ explicit by rewriting the Fisher criterion in the form

$$J(\mathbf{w}) = \frac{\mathbf{w}^{\mathrm{T}}\mathbf{S}_B\mathbf{w}}{\mathbf{w}^{\mathrm{T}}\mathbf{S}_W\mathbf{w}} \qquad \text{(Fisher)}$$

- where $\mathbf{S}_B$ is the **between-class** 散射 matrix and is given by

$$\mathbf{S}_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^{\mathrm{T}}$$

- and $\mathbf{S}_W$ is the **within-class** 散射 matrix, given by

$$\mathbf{S}_W = \sum_{n \in \mathcal{C}_1} (\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^{\mathrm{T}} + \sum_{n \in \mathcal{C}_2} (\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^{\mathrm{T}}.$$

$$\mathbf{S}_\mathbf{b} w = \lambda \mathbf{S}_\mathbf{w} w$$

$$\mathbf{S}_w^{-1}\mathbf{S}_\mathbf{b} w = \lambda w$$

Goal function of the projection

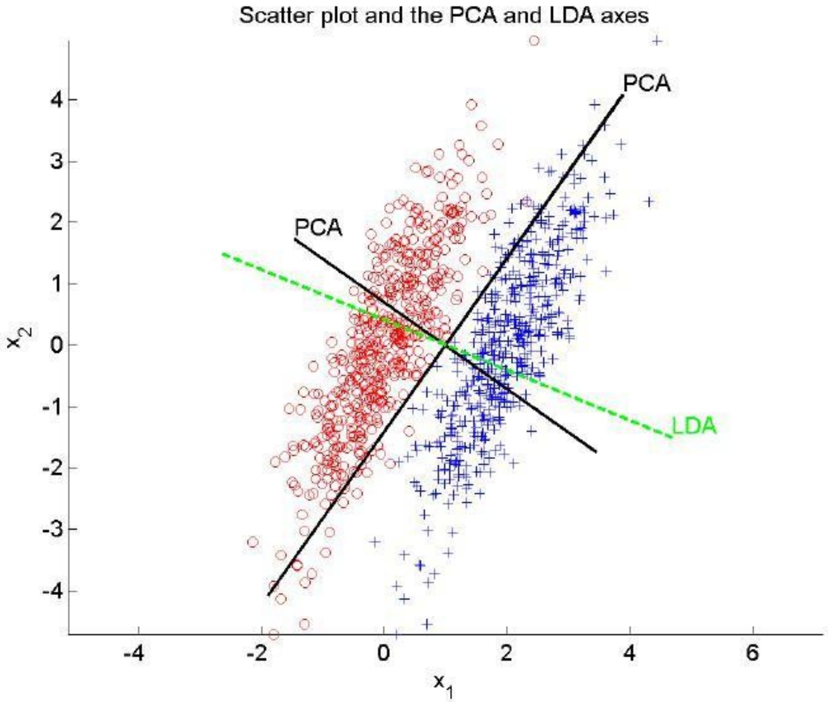
- We express S_W and S_B with multi-class

$$S_B = \sum_{\substack{i,j \\ i \neq j}} \left[(m_i - m_j)(m_i - m_j)^{\mathrm{T}} \right]$$

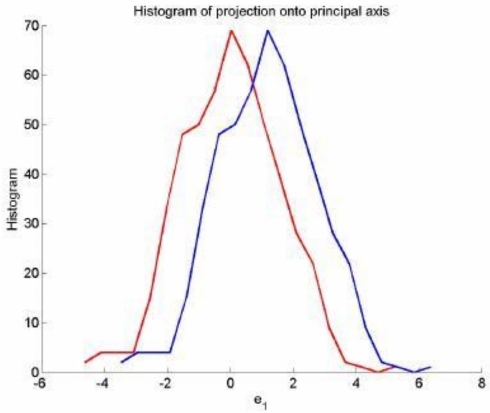
$$S_W = \sum_{k=1}^{K} \left(\sum_{x \in C_k} (x - m_i)(x - m_i)^{\mathrm{T}} \right)$$

- It can be shown that $J(\mathbf{W})$ is maximized when the columns of $\mathbf{W}$ are the eigenvectors of $\mathbf{S}_W^{-1}\mathbf{S}_B$ having the largest eigenvalues.
- Because $\mathbf{S}_B$ is the sum of c matrices of rank one or less, and because only $c-1$ of these are independent, $\mathbf{S}_B$ is of rank $c-1$ or less. Thus, no more than $c-1$ of the eigenvalues are nonzero.
- Once the transformation from the d-dimensional original feature space to a lower dimensional subspace is done using PCA or LDA, parametric or non-parametric methods can be used to train Bayesian classifiers.

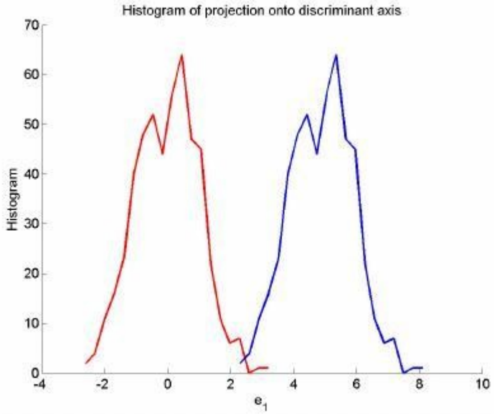
Examples



(a) Scatter plot. Scatter plot and the PCA and LDA axes for a bivariate sample with two classes. Histogram of the projection onto the first LDA axis shows better separation than the projection onto the first PCA axis.

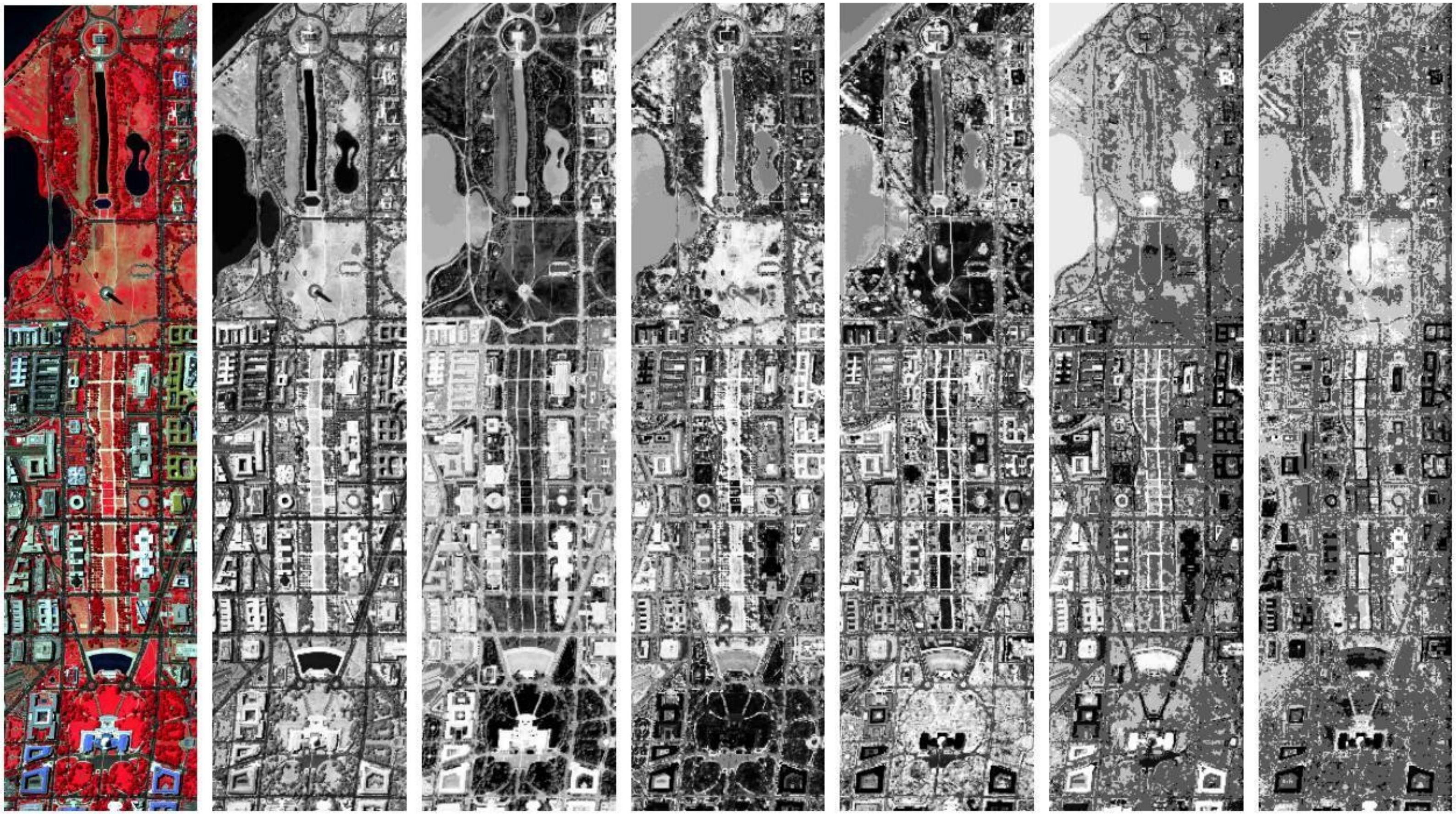


(b) Proj. 1st PCA axis.



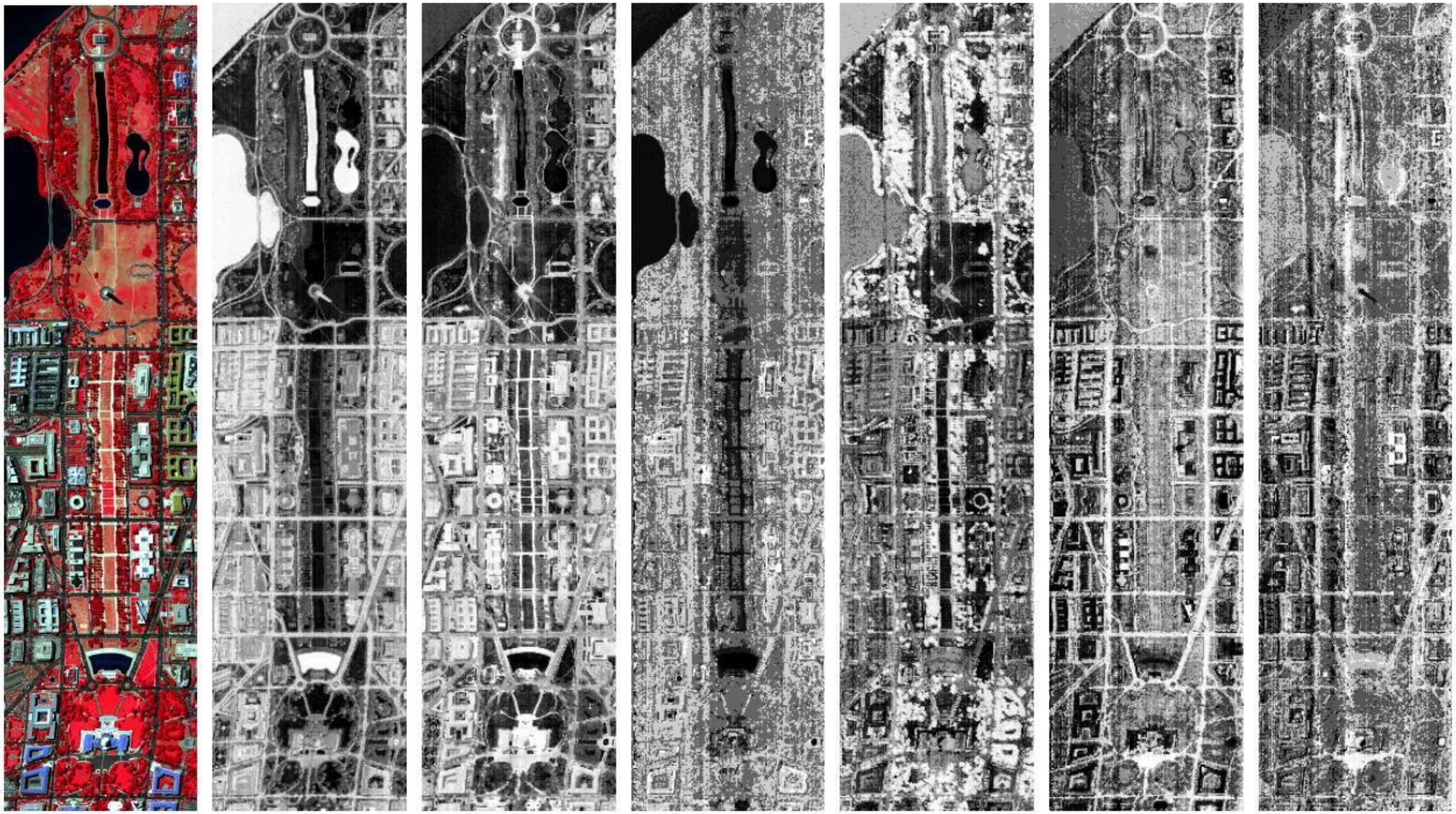
(c) Proj. 1st LDA axis.

Examples



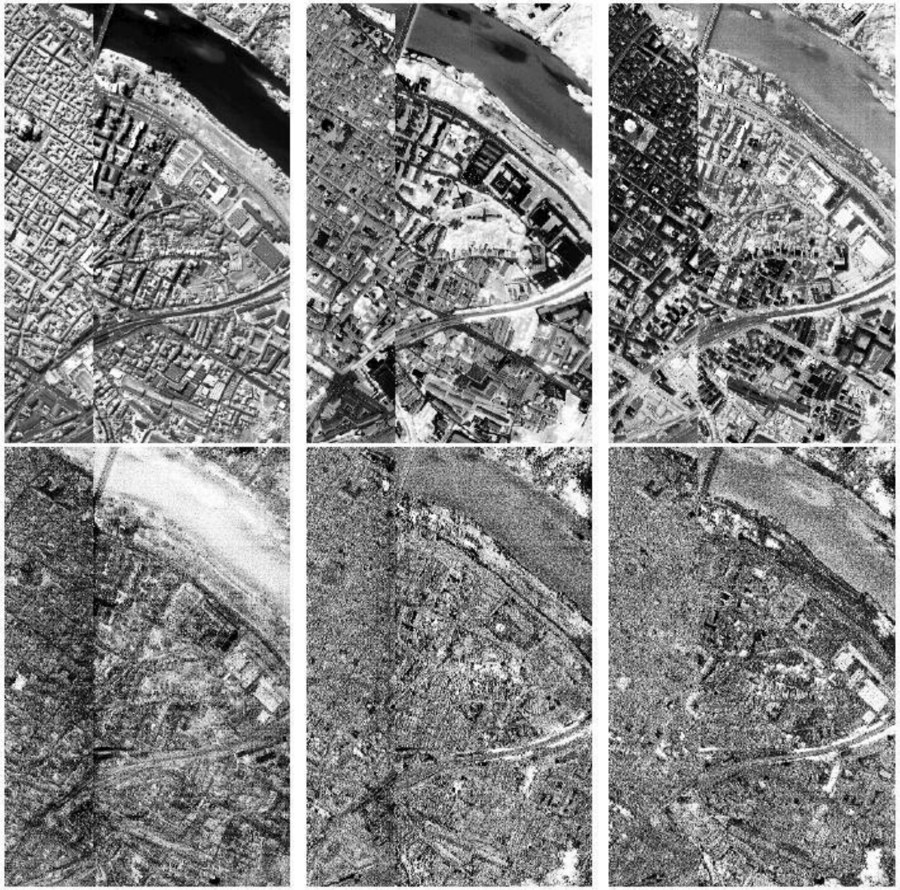
A satellite image and the first six PCA bands (after projection).
Histogram equalization was applied to all images for better visualization.

Examples



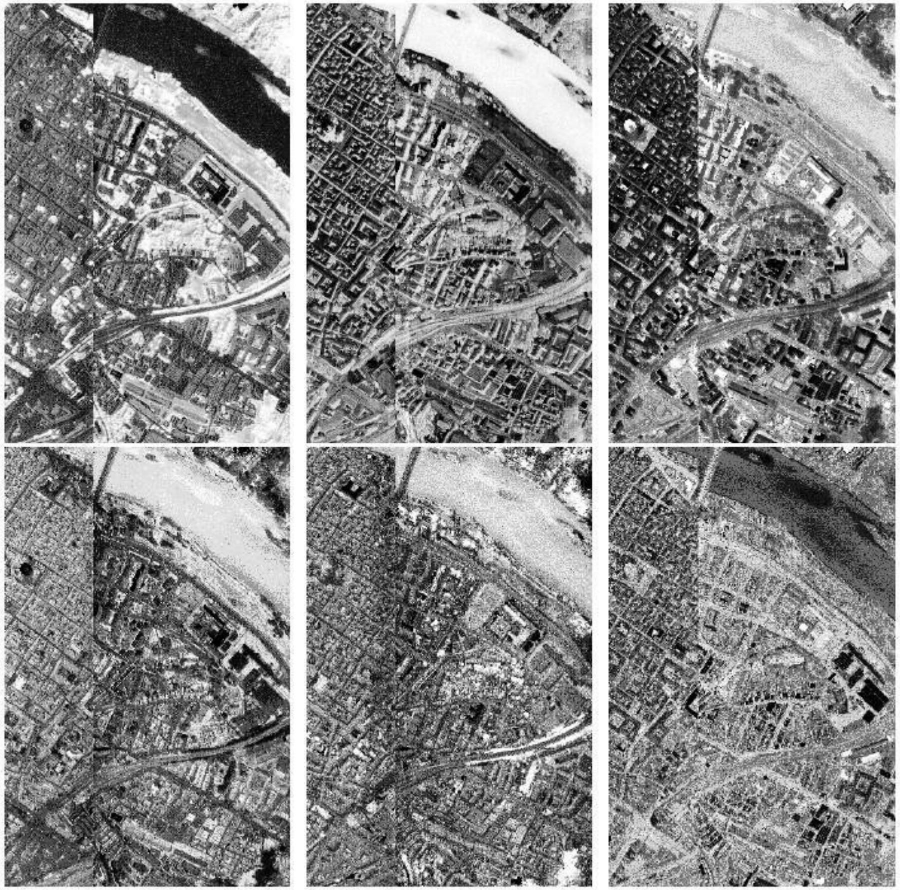
A satellite image and the six **LDA** bands (after projection). Histogram equalization was applied to all images for better visualization.

Examples



A satellite image and the first six PCA bands (after projection). Histogram equalization was applied to all images for better visualization.

Examples

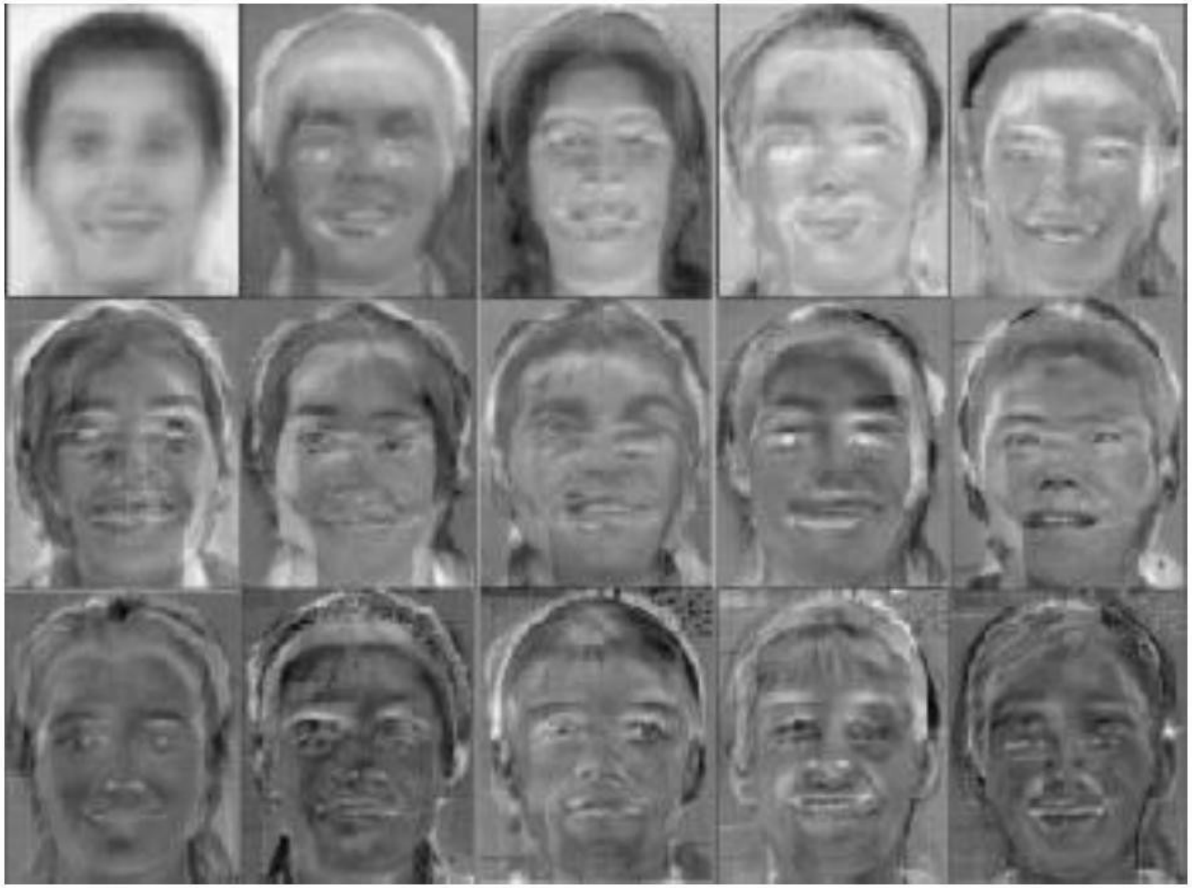


A satellite image and the six **LDA** bands (after projection). Histogram equalization was applied to all images for better visualization.

Examples

Example face images. (Taken from http://www.geop.ubc.ca/CDSST/eigenfaces.html.)

Examples



Eigenvectors (principal axes) of the face images (often referred to as eigenfaces).

Manifold Learning

Isometric Feature Mapping

Isometric Feature Mapping

- The isometric feature mapping (Isomap) algorithm combines the major algorithmic features of PCA and MDS (multi-dimensional scaling)
  - computational efficiency,
  - global optimality, and
  - asymptotic convergence guarantees

  with the flexibility to learn a broad class of nonlinear manifolds.
- The approach seeks to preserve the intrinsic geometry of the data, as captured in the geodesic manifold distances between all pairs of data points.

Isometric Feature Mapping

- The essential point is to estimate the geodesic distance between faraway points, given only input-space distances.
- For neighboring points, input-space distance provides a good approximation.
- For faraway points, geodesic distance can be approximated by adding up a sequence of short hops between neighboring points.
- These approximations are computed efficiently by finding shortest paths in a graph with edges connecting neighboring data points.

Isometric Feature Mapping

- The Isomap algorithm has three steps.
- The first step determines which points are neighbors on the manifold M, based on the distances $d_X(i, j)$ between pairs of points i, j in the input space X.
- A sparse graph G is defined over all data points by connecting points i and j if they are closer than ϵ (ϵ-Isomap) or if i is one of the K nearest neighbors of j (K-Isomap), and the edge weights are set as $d_X(i, j)$.

Isometric Feature Mapping

- In the second step, Isomap estimates the geodesic distances $d_M(i,j)$ between all pairs of points on the manifold M by computing their shortest path distances $d_G(i,j)$ in the graph G.
- The final step applies classical multi-dimensional scaling to the matrix of graph distances $\mathbf{D}_G = \{d_G(i,j)\}$, constructing an embedding of the data in a d-dimensional Euclidean space Y that best preserves the manifold's estimated intrinsic geometry.

Isometric Feature Mapping

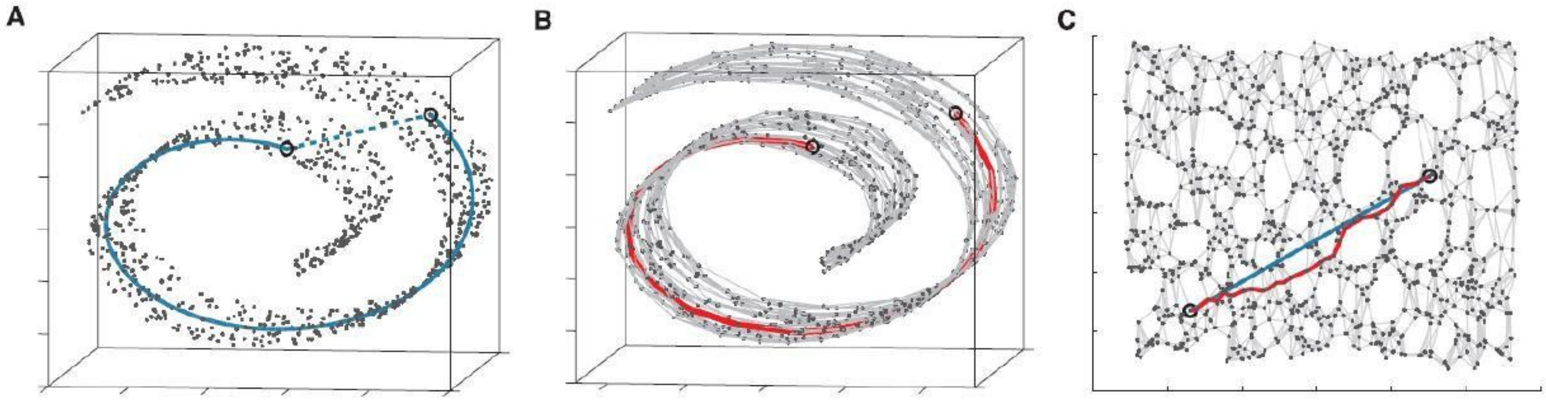
- The coordinate vectors $\mathbf{y}_i$ for points in Y are chosen to minimize the cost function

$$E = \|\tau(\mathbf{D}_G) - \tau(\mathbf{D}_Y)\|_{L^2}$$

where $\mathbf{D}_Y$ denotes the matrix of Euclidean distances $\{d_Y(i,j) = \|\mathbf{y}_i - \mathbf{y}_j\|\}$ and $\|\mathbf{A}\|_{L^2}$ the L^2 matrix norm $\sqrt{\sum_{i,j} A_{i,j}^2}$.

- The τ operator is defined as $\tau(\mathbf{D}) = -\mathbf{HSH}/2$, where $\mathbf{S}$ is the matrix of squared distances $\{S_{ij} = D_{ij}^2\}$, and $\mathbf{H}$ is the centering matrix $\{H_{ij} = \delta_{ij} - 1/N\}$.
- The global minimum of the cost function is achieved by setting the coordinates $\mathbf{y}_i$ to the top d eigenvectors of the matrix $\tau(\mathbf{D}_G)$.

Isometric Feature Mapping

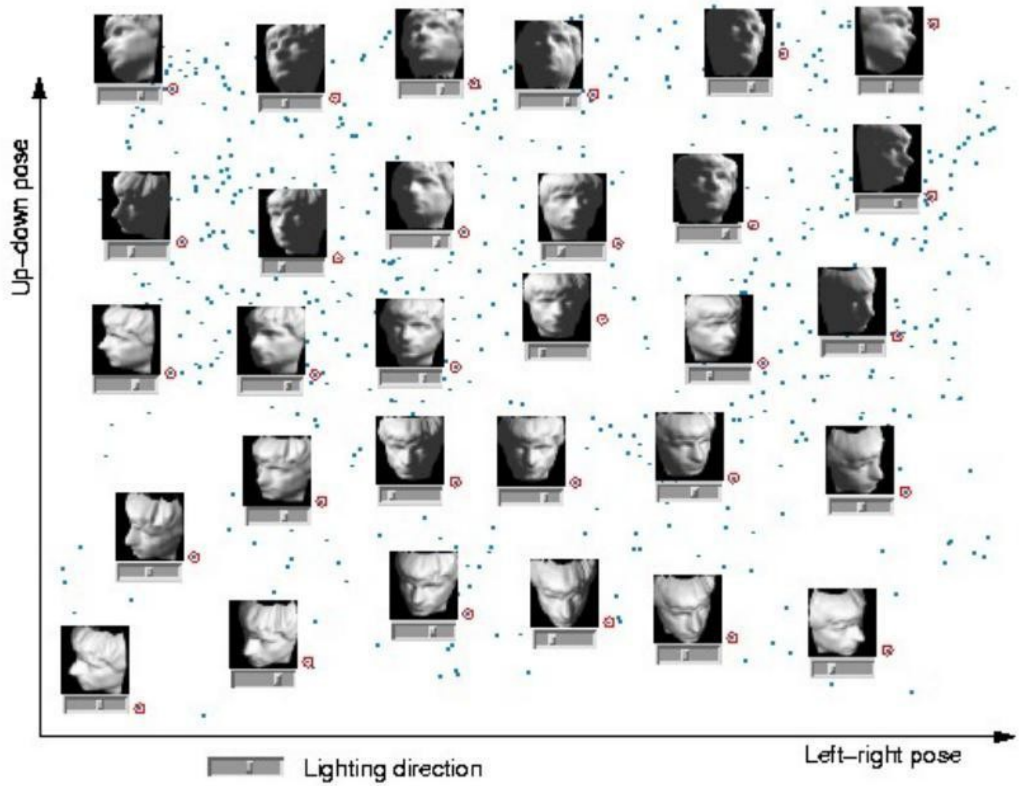


The "Swiss roll" data set. (A) The Euclidean distance between two points in the high-dimensional input space (length of dashed line) may not accurately reflect their intrinsic similarity, as measured by geodesic distance along the low-dimensional manifold (length of solid curve). (B) The neighborhood graph G constructed with $K = 7$ allows an approximation (red segments) to the true geodesic path with the shortest path in G. (C) The two-dimensional embedding recovered by Isomap preserves the shortest path distances in the neighborhood graph. Straight lines in the embedding (blue) now represent cleaner approximations to the true geodesic paths than do the corresponding graph paths (red).

Isometric Feature Mapping

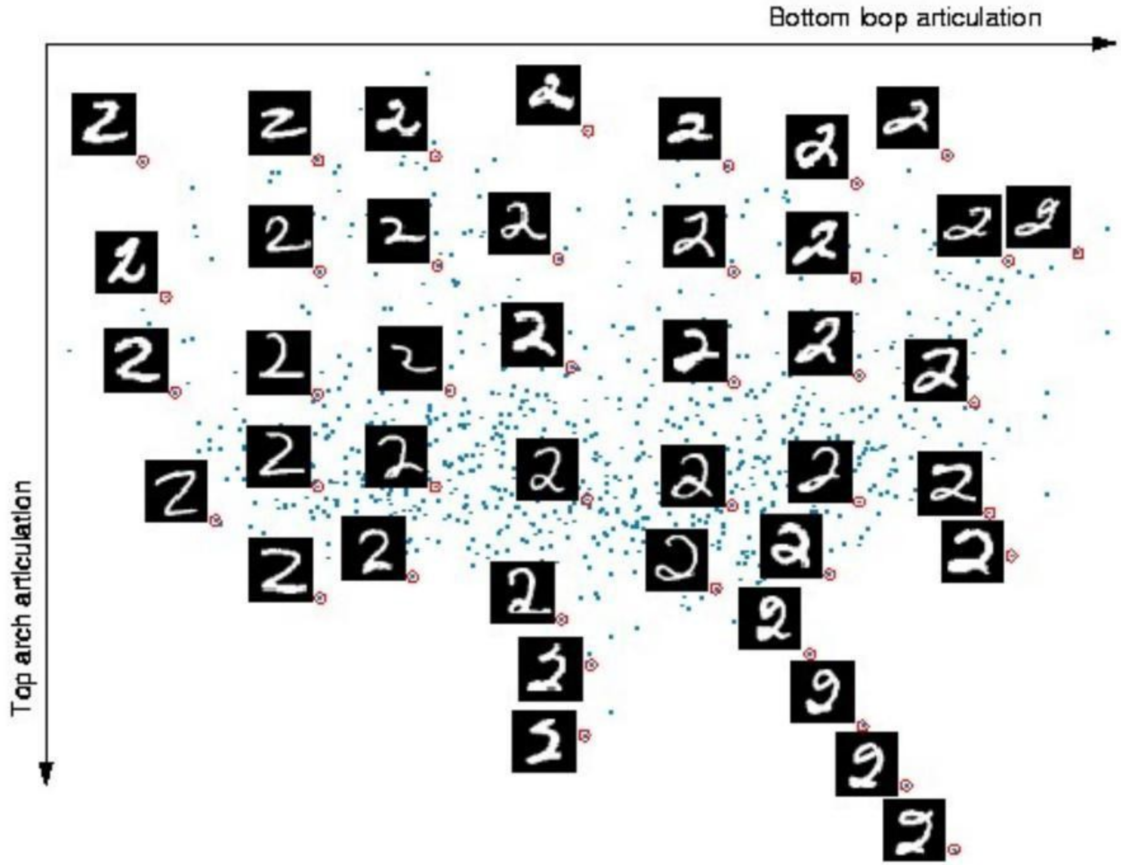
- Pros:
  - A noniterative, polynomial time procedure with a guarantee of global optimality.
  - A guarantee of asymptotic convergence to the true structure for manifolds whose intrinsic geometry is that of a convex region of Euclidean space.
  - Single free parameter (ϵ or K).
- Cons:
  - Sensitive to noise.
  - Computationally expensive (dense matrix eigenreduction).

Examples



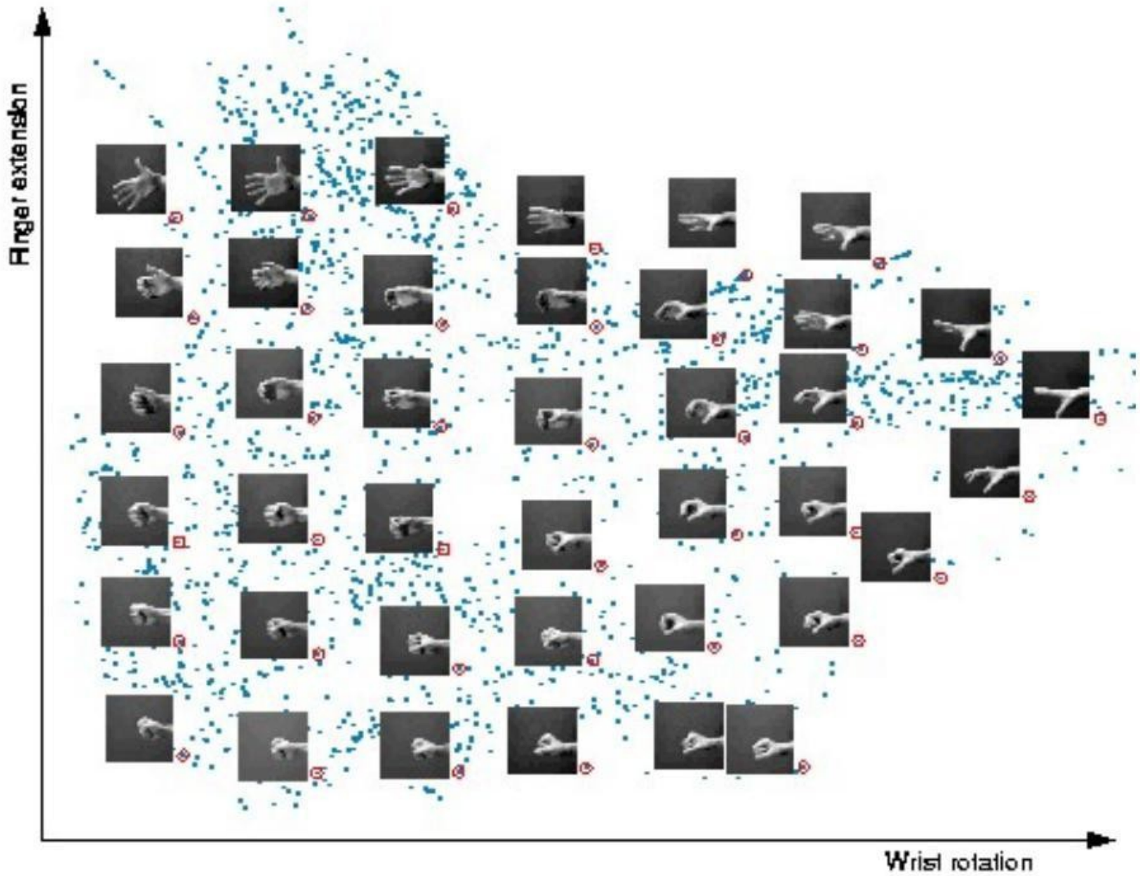
The input consists of 4096-dimensional vectors, representing the brightness values of 64×64 pixel images of a face rendered with different poses and lighting directions. A two-dimensional projection is shown with horizontal sliders (under the images) representing the third dimension. Each coordinate axis of the embedding correlates highly with one degree of freedom underlying the original data: left-right pose (x axis), up-down pose (y axis), and lighting direction (slider position).

Examples



ϵ-Isomap applied to handwritten "2"s. The two most significant dimensions in the Isomap embedding articulate the major features of the "2": bottom loop (x axis) and top arch (y axis).

Examples



Isomap ($K = 6$) applied to 64×64 images of a hand in different configurations. The images were generated by making a series of opening and closing movements of the hand at different wrist orientations.

The recovered coordinate axes map approximately onto the distinct underlying degrees of freedom: wrist rotation (x axis) and finger extension (y axis).

Locally Linear Embedding

Locally Linear Embedding

- The locally linear embedding (LLE) algorithm is based on simple geometric intuitions.
- Suppose that the data consist of N real-valued vectors $\mathbf{x}_i$, each of dimensionality d, sampled from some underlying manifold.
- Provided there is sufficient data (such that the manifold is well-sampled), each data point and its neighbors are expected to lie on or close to a locally linear patch of the manifold.

Locally Linear Embedding

- The local geometry of these patches is characterized by linear coefficients that reconstruct each data point from its neighbors.
- The reconstruction errors are measured by the cost function

$$\epsilon(\mathbf{W}) = \sum_i \left\| \mathbf{x}_i - \sum_j W_{ij} \mathbf{x}_j \right\|^2$$

which adds up the squared distances between all data points and their reconstructions.

- The weights W_{ij} summarize the contribution of the jth data point to the ith reconstruction.

Locally Linear Embedding

- To compute the weights W_{ij}, the cost function is minimized subject to two constraints:
  - each data point $\mathbf{x}_i$ is reconstructed only from its neighbors, enforcing $W_{ij} = 0$ if $\mathbf{x}_j$ does not belong to the set of neighbors of $\mathbf{x}_i$,
  - the rows of the weight matrix sum to one: $\sum_j W_{ij} = 1$.
- The optimal weights W_{ij} subject to these constraints are found by solving a least-squares problem.

Locally Linear Embedding

- The constrained weights that minimize these reconstruction errors obey an important symmetry: for any particular data point, they are invariant to rotations, rescalings, and translations of that data point and its neighbors.
- Suppose that the data lie on or near a smooth nonlinear manifold of lower dimensionality $d' \ll d$.
- By design, the reconstruction weights W_{ij} reflect intrinsic geometric properties of the data that are invariant to such transformations.
- Therefore, their characterization of local geometry in the original data space is expected to be equally valid for local patches on the manifold.

Locally Linear Embedding

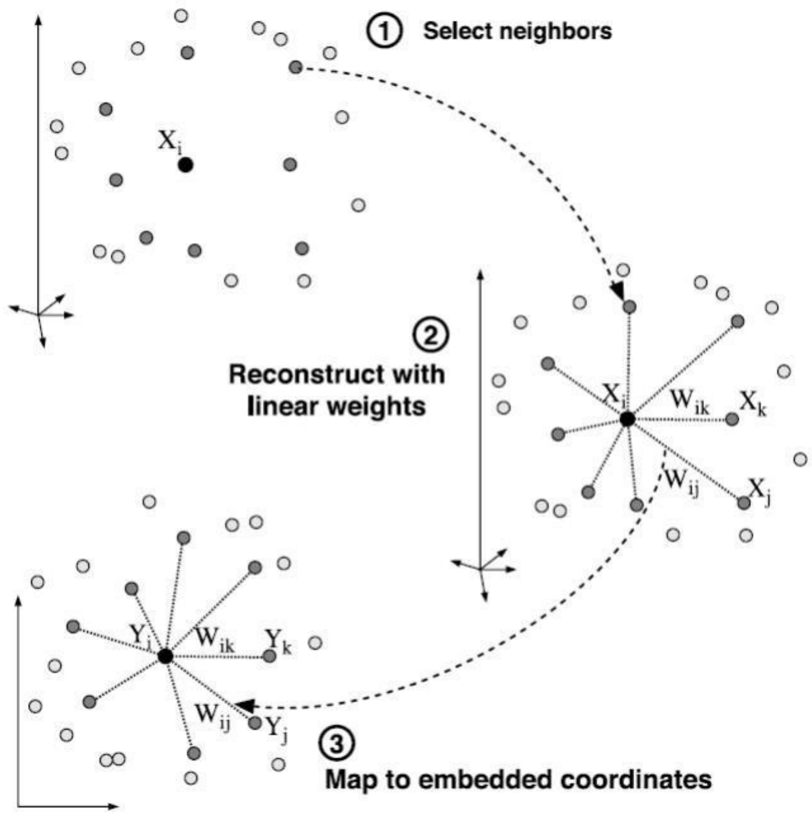
- LLE constructs a neighborhood-preserving mapping based on this idea.
- In the final step of the algorithm, each high-dimensional observation $\mathbf{x}_i$ is mapped to a low-dimensional vector $\mathbf{y}_i$ representing global internal coordinates on the manifold.
- This is done by choosing d'-dimensional coordinates $\mathbf{y}_i$ to minimize the embedding cost function

$$\Phi(\mathbf{Y}) = \sum_i \left\| \mathbf{y}_i - \sum_j W_{ij} \mathbf{y}_j \right\|^2 .$$

Locally Linear Embedding

- This cost function, like the previous one, is based on locally linear reconstruction errors, but here the weights W_{ij} are fixed while optimizing the coordinates $\mathbf{y}_i$.
- The cost function can be minimized by solving a sparse eigenvalue problem whose bottom d' nonzero eigenvectors provide an ordered set of orthogonal coordinates centered on the origin.

Locally Linear Embedding

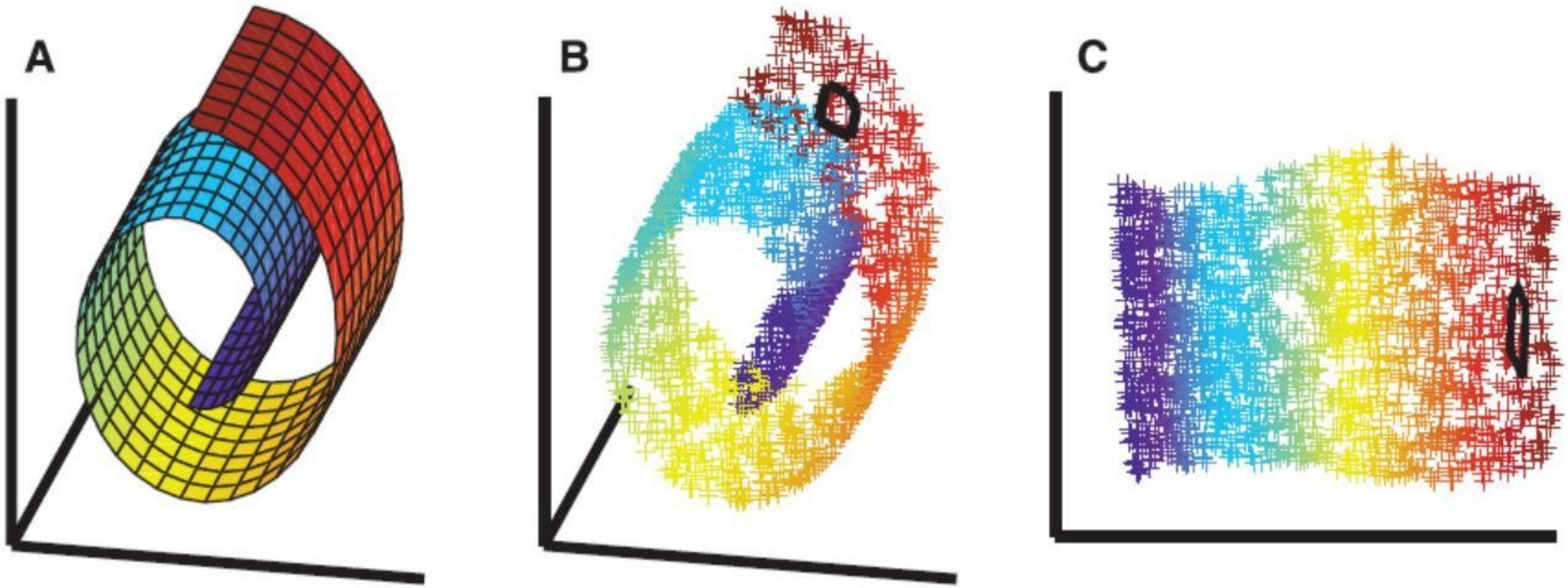


Steps of LLE. (1) Assign neighbors to data point $\mathbf{x}_i$. (2) Compute the weights Wij that best reconstruct $\mathbf{x}_i$ from its neighbors. (3) Compute the low-dimensional embedding vectors $\mathbf{y}_i$ best reconstructed by W_{ij}.

Locally Linear Embedding

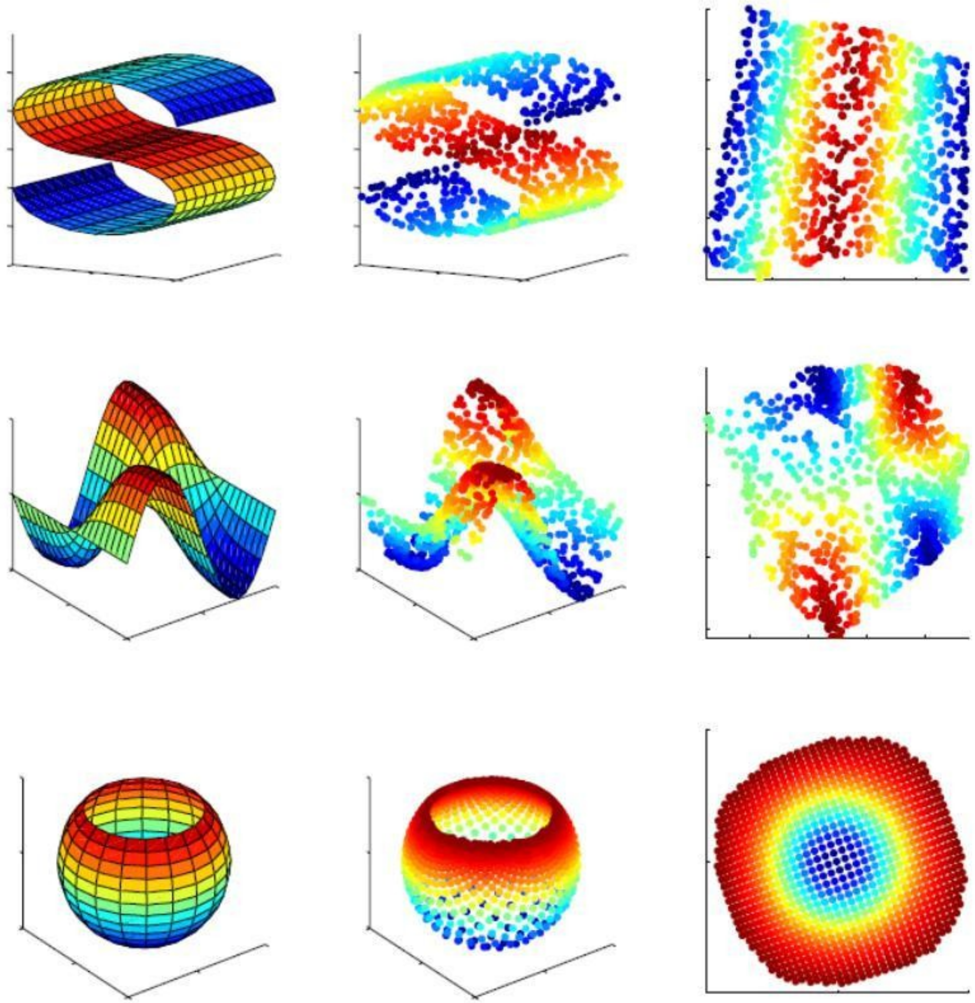
- Pros:
  - Globally optimal result.
  - Single free parameter (number of neighbors, K).
  - Simple linear algebra operations using sparse matrices.
- Cons:
  - Sensitive to noise.
  - No theoretical guarantees.

Examples



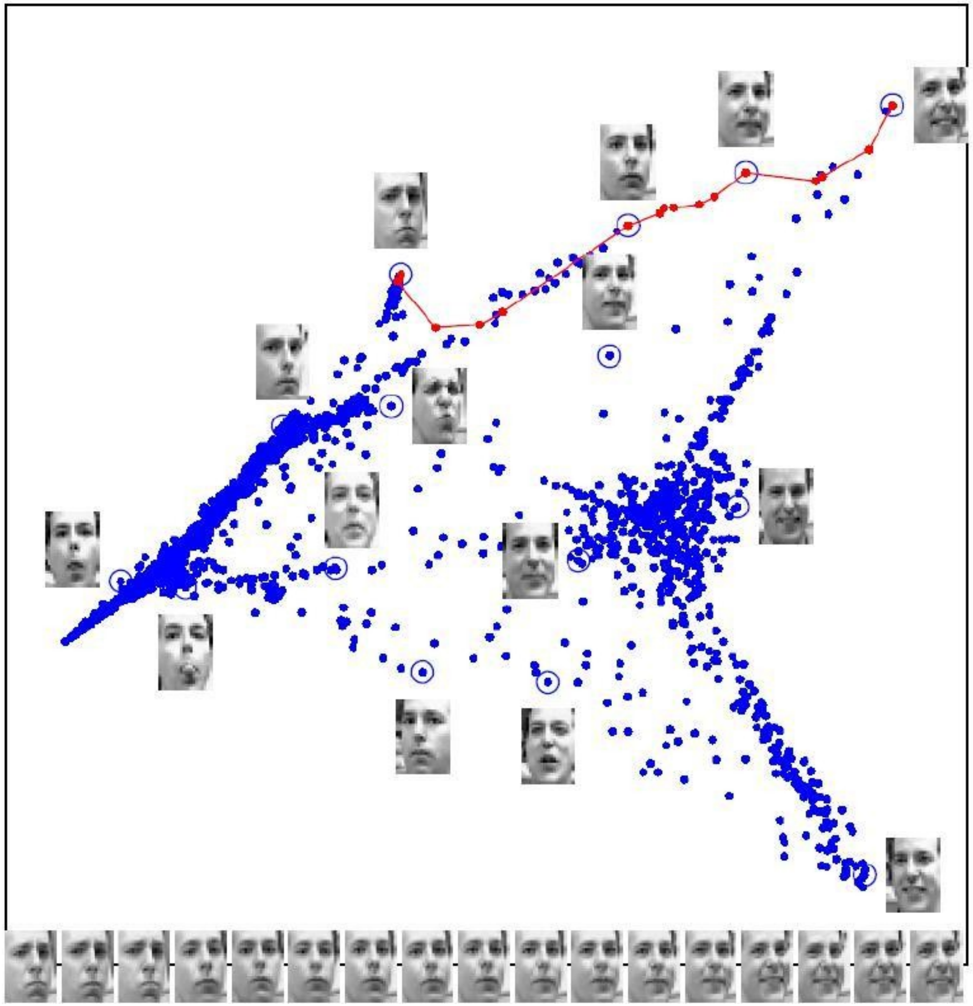
The "Swiss roll" data set. The color coding illustrates the neighborhood-preserving mapping discovered by LLE. Black outlines in (B) and (C) show the neighborhood of a single point.

Examples



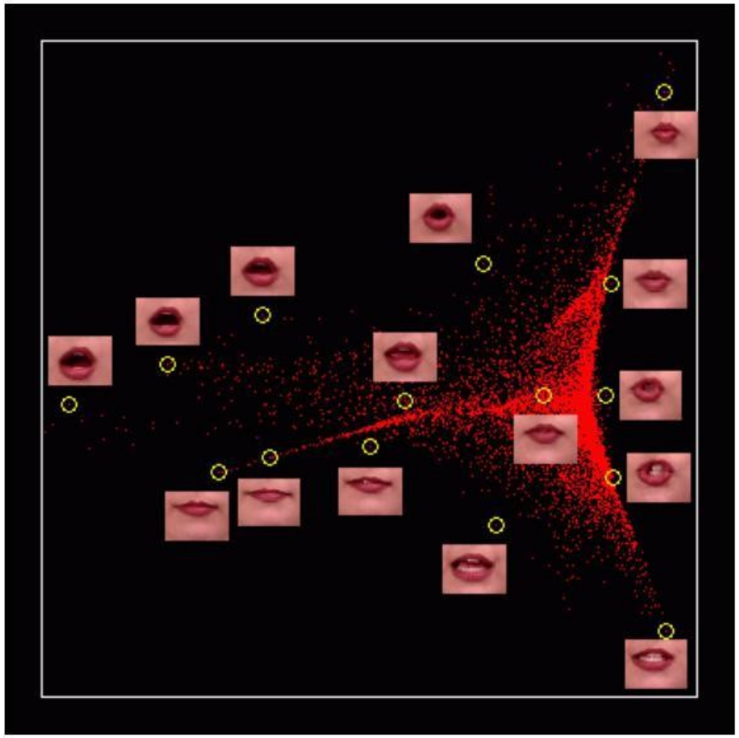
Other examples for three-dimensional data sampled from two dimensional manifolds.

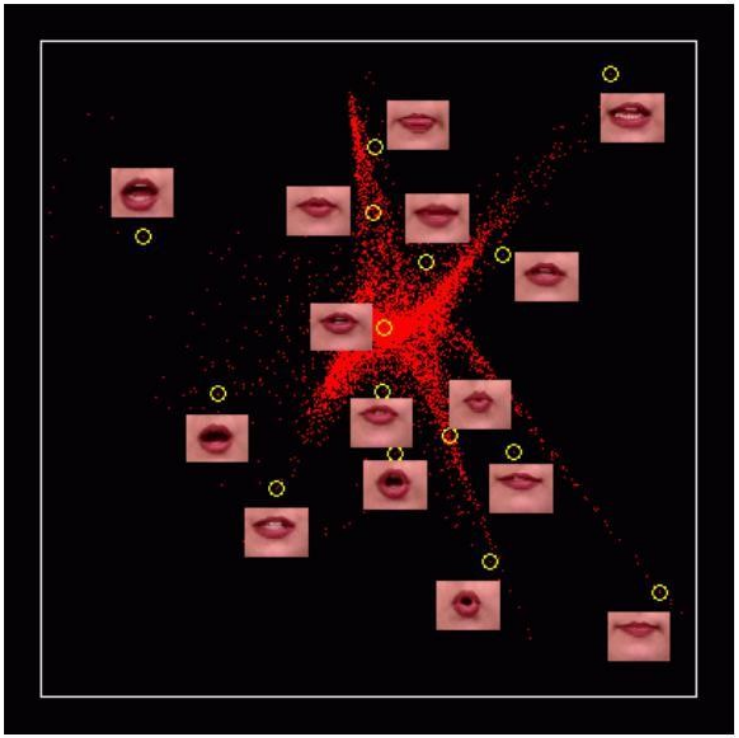
Examples



Images of faces, digitized at 20×28 pixels, mapped into the embedding space described by the first two coordinates of LLE. The bottom images correspond to points along the top-right path (linked by solid red line), illustrating one particular mode of variability in pose and expression.

Examples





Images of lips, mapped into the embedding space described by the first two coordinates of LLE.

XIDIAN UNIVERSITY

Stochastic Neighbor Embedding

Stochastic Neighbor Embedding

- Nonlinear dimensionality reduction methods are especially suited for projecting to 2D or 3D (thus for visualization)
- Stochastic Neighbor Embedding or SNE (Hinton and Roweis, 2002) uses the idea of preserving probabilistically defined neighborhoods
- SNE, for each point i, define the probability of point j being its neighbor as

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma^2)}{\sum_{k\neq i}\exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma^2)} \quad q_{j|i} = \frac{\exp(-\|\mathbf{z}_i - \mathbf{z}_j\|^2/2\sigma^2)}{\sum_{k\neq i}\exp(-\|\mathbf{z}_i - \mathbf{z}_k\|^2/2\sigma^2)}$$

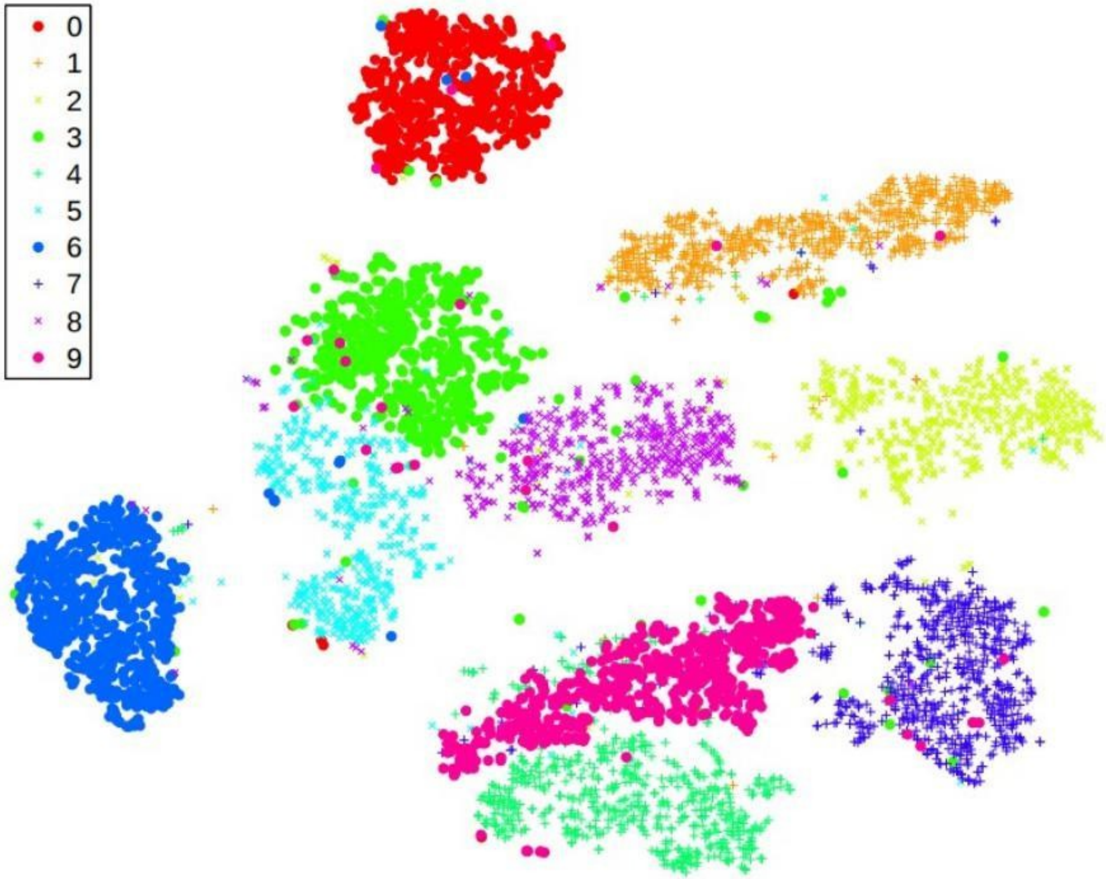
- The p's denotes probabilities in original space, the q's denote prob. in embedded space
- SNE learns $\mathbf{z}_i$'s such that distribution P and Q is as close as possible by minimizing $\mathrm{KL}(P\|Q)$.

- t-SNE (van der Maaten and Hinton, 2008) offers a couple of improvements to original SNE
  - Learns $\mathbf{z}_i$'s by minimizing **symmetric KL divergence**
  - Uses **Student** t **distribution** instead of Gaussian for defining $q_{j|i}$

Visualization

- Especially useful for visualizing data by projecting into 2D or 3D

SNE and t-SNE



Result of visualizing MNIST digits data in 2D (Figure from van der Maaten and Hinton, 2008)

Metric Learning

Metric Learning

- The main purpose of dimensionality reduction is to find an adequate low-dimensional space, in which learning algorithms can obtain performance better than the original space.
- In fact, each space defines a distance metric on features of corresponding samples.
- To find the adequate space is to find the adequate distance metric.
- So why not try to "learn" an adequate metric? — This is the motivation of **metric learning**.

- To learn the metric, we first need a convenient representation of the distance metric.
- Previously some expressions of distance metrics have been introduced. However they are in **fixed** form without learnable parameters.
- Assume that different attributes are with different importance, we can introduce attributes related weights, thus

$$\begin{aligned}\mathrm{dist}_{\mathrm{wed}}(\mathbf{x}_i,\mathbf{x}_j) &= w_1(x_{i1}-x_{j1})^2 + w_2(x_{i2}-x_{j2})^2 + \cdots + w_d(x_{id}-x_{jd})^2 \\ &= (\mathbf{x}_i-\mathbf{x}_j)^T\mathbf{W}(\mathbf{x}_i-\mathbf{x}_j)\end{aligned},$$

where $w_l \geq 0$, $\mathbf{W} = \mathrm{diag}(\mathbf{w})$ is a diagonal matrix with $W_{ll} = w_l$.

Metric Learning

- The above $\mathbf{W}$ is learnable.
- The weight matrix $\mathbf{W}$ is diagonal which means attributes are uncorrelated.
- But in practical problems, attributes are often correlated.
- So we replace $\mathbf{W}$ with a more general positive semidefinite matrix $\mathbf{M}$, then we get the **Mahalanobis distance**:

$$\mathrm{dist}_{\mathrm{mah}}(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{M} (\mathbf{x}_i - \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2,$$

  where $\mathbf{M}$ is also called the **metric matrix**, and metric learning is to learn matrix $\mathbf{M}$.
- To preserve both the non-negative and symmetric properties of being distances, the matrix $\mathbf{M}$ must be a positive semidefinite matrix, i.e., there must exist a group orthogonal basis such that $\mathbf{M} = \mathbf{P}\mathbf{P}^T$.

- We need to set a goal to learn $\mathbf{M}$.
- Suppose that we expect to improve the performance of the nearest neighbor classifier, then the matrix $\mathbf{M}$ can be embedded to the performance measure directly.
- By optimizing the peformance we can obtain $\mathbf{M}$.
- Discriminant in neighborhood classifiers often employs major voting,
  - samples inside the neighborhood vote for one,
  - while samples outside the neighborhood vote for zero.

- We can use a probabilistic voting approach. For any sample $\mathbf{x}_j$, its probability of effects on the classification of sample $\mathbf{x}_i$ is

$$p_{ij} = \frac{\exp\Big(-\|\mathbf{x}_i - \mathbf{x}_j\|_M^2\Big)}{\sum_l \exp\Big(-\|\mathbf{x}_i - \mathbf{x}_l\|_M^2\Big)},$$

- when $i = j$, p_{ij} gets maximized.
- Obviously, the effect of $\mathbf{x}_j$ to $\mathbf{x}_i$ decreases along with the increment of the distance between them.

- If the goal is to maximize the correction ratio of leave-one-out (LOO), we can compute the ratio:

$$p_i = \sum_{j \in \Omega_i} p_{ij},$$

  - where Ω_i denotes the set of indices of samples which are with a label same to sample $\mathbf{x}_i$.
- Thus the overall correction ratio of LOO is

$$\sum_{i=1}^{m} p_i = \sum_{i=1}^{m} \sum_{j \in \Omega_i} p_{ij}.$$

- Combine previous equations and considering $\mathbf{M} = \mathbf{P}\mathbf{P}^T$, the neighborhood component analysis, NCA, (Goldberger *et al.*, 2005) is

$$\min_{\mathbf{P}} 1 - \sum_{i=1}^{m} \sum_{j \in \Omega_i} \frac{\exp\Big(- \|\mathbf{P}^T\mathbf{x}_i - \mathbf{P}^T\mathbf{x}_j\|_2^2 \Big)}{\sum_l \exp\Big(- \|\mathbf{P}^T\mathbf{x}_i - \mathbf{P}^T\mathbf{x}_l\|_2^2 \Big)}.$$

- Solve the above equation will get the metric matrix $\mathbf{M}$ which maximizes the LOO correction ratio of the nearest neighbor classifier.

- In fact, we can also introduce domain knowledge into metric learning.
- If some samples are similar and some others are not are known, we can define "must-link" set $\mathcal{M}$ and "cannot-link" set $\mathcal{C}$ as constraints.
  - where $(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{M}$ denotes $\mathbf{x}_i$ and $\mathbf{x}_j$ are similar,
  - and $(\mathbf{x}_i, \mathbf{x}_j) \in \mathcal{C}$ denotes $\mathbf{x}_i$ and $\mathbf{x}_j$ are NOT similar.
- We expect that the distance between similar samples are small, while large for dissimilar samples.

- Then the adequate metric matrix $\mathbf{M}$ can be obtained by solving the following convex optimization (Xing *et al.*, 2003):

$$\begin{aligned}\min_{\mathbf{M}} \quad & \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{M}} \|\mathbf{x}_i-\mathbf{x}_j\|_{\mathbf{M}}^2 \\ \text{s.t.} \quad & \sum_{(\mathbf{x}_i,\mathbf{x}_j)\in\mathcal{C}} \|\mathbf{x}_i-\mathbf{x}_j\|_{\mathbf{M}}^2 \geq 1, \\ & \mathbf{M}\succeq 0,\end{aligned}$$

  where $\mathbf{M}\succeq 0$ indicates $\mathbf{M}$ must be positive semidefinite.
- Different metric learning methods are for different goals.
- We can learn a low-rank matrix $\mathbf{M}$ for dimensionality reduction.

Summary

Summary of Dimensionality Reduction

Method	Property
Principal Components Analysis (PCA)	Linear Map; fast; eigenvector-based
Linear Discriminant Analysis	Supervised Linear Map; fast, eigenvector-based
Projection Pursuit	Linear Map; Iterative; non-Gaussian
Independent Components Analysis (ICA)	Linear Map; Iterative; non-Gaussian
PCA Network	Linear map; iterative

Summary of Dimensionality Reduction

Method	Property
Kernel PCA	Nonlinear map; eigenvector-based
Nonlinear PCA	Linear map; non-Gaussian criterion; usually iterative
Nonlinear auto-associative network	Nonlinear map; non-Gaussian criterion; iterative
Multidimensional scaling (MDS)	Nonlinear map; iterative
Sammon's projection	Nonlinear map; iterative
Self-Organizing Map (SOM)	Nonlinear; iterative